

Tangent-Space Methods for Nonlinear Control and Biomechanics

Volume III: Biomechanics

From Rigid Bodies to Biological Systems

Preface

The first two volumes of *Tangent-Space Methods for Nonlinear Control and Biomechanics* developed the mathematical language of tangent spaces, contraction theory, and trajectory-centric control with the implicit assumption that the systems under study are *rigid bodies*—that joints are perfect revolute, that links are perfectly stiff, and that actuators produce torques on command. In biological systems, *none of these assumptions hold*.

This volume confronts reality. Biological tissues are viscoelastic, not rigid. Joints permit coupled translations and rotations, not single-axis pivots. Muscles produce force through a complex cascade from neural excitation through calcium dynamics to cross-bridge cycling, with force depending on length, velocity, and activation history. Tendons store and release elastic energy. Ligaments constrain motion in ways that change with loading rate.

And yet, the mathematical framework of Volumes 0–II is not abandoned—it is *extended*. The mass matrix $M(\mathbf{q})$ still exists, but it now includes the effective inertia of muscles and the compliance of tendons. The configuration space $\mathcal{Q}$ is still a manifold, but its geometry is richer because biological joints are not simple revolute. The control input $\mathbf{u}$ is no longer a torque but a neural excitation signal, and the mapping from excitation to torque passes through the muscle model.

The philosophy remains unchanged: mathematics *driven by physical reality*, with every abstraction justified by measurement and every model validated against experiment. Where engineering systems are designed to be simple, biological systems have evolved to be *robust*. Understanding why requires the tools of this volume.

Contents

Preface	i
Nomenclature	v
1 How Biology Differs from Engineering	1
1.1 The Rigid-Body Assumption and Its Limits	1
1.2 Why the Differences Matter for Control	2
1.2.1 Compliance Creates Hidden Dynamics	2
1.2.2 Muscle Force Production Is State-Dependent	3
1.2.3 Redundancy Is a Feature, Not a Bug	4
1.3 Modeling Strategies	4
1.3.1 Strategy 1: Rigid-Body with Torque Actuators	4
1.3.2 Strategy 2: Rigid-Body with Muscle Actuators	5
1.3.3 Strategy 3: Flexible Multi-Body with Muscle Actuators	5
1.4 A Quantitative Tour of the Human Body	6
1.4.1 Segment Inertial Properties	7
1.4.2 Joint Ranges of Motion	8
1.4.3 Muscle Properties	8
1.5 Connecting to the Control Framework	9
2 Musculoskeletal Modeling Conventions	12
2.1 Anatomical Reference Frames	12
2.1.1 The Anatomical Position	12
2.1.2 Segment Reference Frames	12
2.1.3 Connection to Screw Theory	13
2.2 Joint Coordinate Systems	13
2.2.1 The Grood–Suntay Convention for the Knee	13
2.2.2 Multi-Representation Joint Angles	15
2.3 Body Segment Parameters	16
2.3.1 Regression Equations	16
2.3.2 Computing the Mass Matrix	16
3 Muscle Models	18
3.1 The Hill-Type Muscle Model	18
3.1.1 Model Architecture	18
3.1.2 The Force-Length Relationship	18
3.1.3 The Force-Velocity Relationship	22
3.1.4 Activation Dynamics	22
3.2 Tendon Mechanics	23
3.3 Musculotendon Equilibrium	23
3.4 The Complete Hill Model as a Dynamical System	24

4	Joint Kinematics and Soft Tissue	26
4.1	Beyond Ideal Joints	26
4.1.1	The Instantaneous Screw Axis	26
4.1.2	The Knee: Rolling, Sliding, and Coupled Rotation	27
4.1.3	The Shoulder: The Most Complex Joint	28
4.2	Ligament Models	29
4.3	Spinal Mechanics	29
5	Multi-Body Dynamics of Biological Chains	30
5.1	Open-Chain vs. Closed-Chain Biomechanics	30
5.1.1	Equations of Motion for Biological Chains	30
5.1.2	Bi-articular Muscles	31
5.1.3	Co-Contraction and Joint Stiffness	32
5.2	The Mass Matrix of the Human Body	33
6	Inverse Problems in Biomechanics	34
6.1	The Inverse Dynamics Pipeline	34
6.1.1	Inverse Kinematics	34
6.1.2	Bottom-Up vs. Top-Down Inverse Dynamics	35
6.2	The Muscle Redundancy Problem	35
6.2.1	Static Optimization	36
6.2.2	Computed Muscle Control (CMC)	36
6.3	Parameter Estimation	36
6.3.1	Bayesian Framework	36
7	Experimental Methods	38
7.1	Motion Capture Systems	38
7.1.1	Marker-Based Systems	38
7.1.2	Markerless Systems	38
7.2	Electromyography (EMG)	39
7.2.1	EMG Processing	40
7.3	Force Plates	40
7.4	Inertial Measurement Units (IMUs)	40
7.5	Data Synchronization	40
8	Inference on Biological Systems	42
8.1	Parameter Identification from Motion Data	42
8.1.1	Maximum Likelihood Estimation	42
8.1.2	Bayesian Estimation with Physical Constraints	42
8.2	System Identification for Biological Systems	44
8.3	Machine Learning Approaches	44
9	Deformable Bodies and Continuum Mechanics	46
9.1	When Rigid-Body Assumptions Fail	46
9.2	Beam Models for Long Bones and Shafts	46
9.2.1	Modal Analysis	46

9.2.2	Golf Club Shaft Flexibility	47
9.3	Finite Element Methods (Overview)	48
10	Applications of Control Theory to Biological Systems	50
10.1	Forward Dynamics Simulation	50
10.2	Predictive Simulation	52
10.3	Prosthetics and Exoskeleton Control	52
10.4	Sport Biomechanics Applications	53
10.4.1	The Golf Swing as a Biomechanical System	53
10.5	Synthesis: From Theory to Application	53

Nomenclature

General Conventions

- Scalars are lowercase or Greek letters (e.g., m, t, θ).
- Vectors are bold lowercase (e.g., $\mathbf{x}, \mathbf{u}, \mathbf{q}$).
- Matrices are bold uppercase (e.g., $\mathbf{A}, \mathbf{B}, \mathbf{M}, \mathbf{J}$).
- Expected values and sets are denoted by blackboard bold or script (e.g., $\mathbb{E}, \mathcal{S}$).

State and Kinematics

$\mathbf{q} \in \mathbb{R}^n$ Joint configuration vector (generalized coordinates).

$\dot{\mathbf{q}} \in \mathbb{R}^n$ Joint velocity vector (generalized velocities).

$\ddot{\mathbf{q}} \in \mathbb{R}^n$ Joint acceleration vector.

$\mathbf{x} \in \mathbb{R}^{n_x}$ System state vector; commonly $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$ for mechanical systems.

$\mathbf{u} \in \mathbb{R}^m$ Control input vector (e.g., joint torques, muscle activations).

$\boldsymbol{\tau} \in \mathbb{R}^n$ Generalized forces/torques acting on the joints.

$t \in \mathbb{R}$ Time.

Inertia and Dynamics

$\mathbf{M}(\mathbf{q})$ Mass (inertia) matrix, symmetric positive definite.

$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ Coriolis and centrifugal force matrix.

$\mathbf{g}(\mathbf{q})$ Gravity vector.

$\mathbf{J}(\mathbf{q})$ Jacobian matrix relating joint velocities to task-space velocities ($\dot{\mathbf{p}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$).

Control and Optimization

$\mathbf{A}_k = \frac{\partial f}{\partial \mathbf{x}}$ Local Jacobian matrix of the state dynamics.

$\mathbf{B}_k = \frac{\partial f}{\partial \mathbf{u}}$ Local Jacobian matrix of the control input.

$V(\mathbf{x})$ or $\mathcal{V}$ Value function or Lyapunov function.

$\mathbf{Q}$ State penalty matrix in LQR/DDP.

R Control penalty matrix in LQR/DDP.

K Feedback gain matrix ($\delta \mathbf{u} = -\mathbf{K}\delta \mathbf{x}$).

γ A trajectory or flow, mapping time and initial conditions to states.

Biomechanics and Motor Control

$a_i \in [0, 1]$ Muscle activation level.

ℓ_i^M Muscle fiber length.

v_i^M Muscle fiber contraction velocity.

ℓ_i^{MT} Musculotendon unit length.

$\mathbf{F}_{\text{muscle}}$ Force generated by muscles.

W Muscle synergy matrix (weights).

$\mathbf{c}(t)$ Muscle synergy activation vector over time.

Geometry and Manifolds

$SE(3)$ Special Euclidean group of rigid body motions.

$SO(3)$ Special Orthogonal group of 3D rotations.

$\mathfrak{se}(3)$ Lie algebra of $SE(3)$, space of spatial twists (velocities).

$\mathcal{M}$ A smooth manifold.

$T_{\mathbf{x}}\mathcal{M}$ Tangent space at point $\mathbf{x}$.

Chapter 1

How Biology Differs from Engineering

Nature does not build machines. It grows organisms. The engineer's model of the body is a useful fiction—but it is fiction.

— Adapted from D'Arcy Wentworth Thompson

1.1 The Rigid-Body Assumption and Its Limits

Every model in Volumes 0–II rested on a fundamental assumption: the moving system consists of *rigid bodies* connected by *ideal joints*. A rigid body has a fixed mass distribution; its shape does not change under load. An ideal revolute joint constrains five of six relative degrees of freedom, permitting only rotation about a single axis. These assumptions are extraordinarily productive for engineering systems—robot arms, spacecraft, mechanisms—where components are designed to approximate rigidity.

In biological systems, these assumptions fail at every level:

- (i) **Bones are not rigid.** Long bones (femur, humerus, tibia) have measurable compliance under dynamic loading. A golf club shaft deflects by several centimeters and several degrees during a swing; a human femur deflects by millimeters during running. The deflection is small in absolute terms but can have significant effects on end-effector accuracy.
- (ii) **Joints are not revolute.** The knee, for example, combines rolling, sliding, and rotation in a coupled manner that changes with flexion angle. The glenohumeral (shoulder) joint permits motion in three rotational axes but also allows several millimeters of translation. The intervertebral joints of the spine have six degrees of freedom each, all coupled through the disc and ligament mechanics.
- (iii) **Actuators are not torque sources.** Muscles produce force, not torque. Force is generated through the contraction of sarcomeres, transmitted through tendons, and converted to joint torque via moment arms that vary with joint angle. The force a muscle can produce depends on its length, its contraction velocity, and its activation history—dependencies that have no analog in electric motors.

- (iv) **Sensors are noisy, delayed, and multimodal.** Proprioceptors (muscle spindles, Golgi tendon organs, joint receptors) provide state information with significant noise and delays of 30–100 ms. The central nervous system must fuse information from proprioceptive, visual, and vestibular sources to estimate the body’s state.
- (v) **The system adapts.** Unlike a robot, the human body changes over time—muscles fatigue, connective tissue remodels, neural pathways strengthen or weaken. A complete model must account for adaptation on timescales from seconds (fatigue) to years (training).

Biology vs. Engineering 1.1. The Fundamental Differences

Property	Engineering System	Biological System
Links	Rigid bodies	Viscoelastic, deformable
Joints	1-DOF revolute/prismatic	Multi-DOF, coupled, with laxity
Actuators	Motors (torque on command)	Muscles (force via excitation-contraction)
Transmission	Gears, belts (rigid)	Tendons (elastic, energy-storing)
Sensors	Encoders (precise, fast)	Proprioceptors (noisy, delayed)
Parameters	Fixed (by design)	Variable (fatigue, adaptation, growth)
Redundancy	Minimal (cost-driven)	Massive (evolution-driven)
Failure mode	Brittle (sudden)	Graceful (degradation)

1.2 Why the Differences Matter for Control

The differences enumerated above are not merely academic. They fundamentally change the control problem in ways that Volumes I and II must now accommodate.

1.2.1 Compliance Creates Hidden Dynamics

When a link is compliant, its deformation adds internal degrees of freedom to the system. A golf club shaft, modeled as rigid, has one degree of freedom (the grip angle). Modeled as a flexible beam, it has *infinitely many* degrees of freedom, though practical models truncate to the first 2–3 bending modes. The equation of motion becomes:

$$\underbrace{M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + g(\mathbf{q})}_{\text{rigid-body dynamics}} + \underbrace{K_{\text{stiff}}\mathbf{q}_{\text{flex}} + D_{\text{damp}}\dot{\mathbf{q}}_{\text{flex}}}_{\text{flexibility terms}} = \boldsymbol{\tau}, \quad (1.1)$$

where $\mathbf{q}_{\text{flex}}$ represents the modal coordinates of the flexible modes, K_{stiff} is the stiffness matrix, and D_{damp} is the structural damping matrix.

Biomechanical Principle 1.1. The Timescale Separation Principle

In many biological systems, the flexible dynamics are *fast* relative to the rigid-body dynamics. If the natural frequency of the flexible modes is much higher than the bandwidth of the controller, the flexibility can be modeled as a quasi-static compliance rather than a dynamic mode. Specifically, if the k -th flexible mode has natural frequency ω_k and the controller bandwidth is ω_c , then:

- $\omega_k \gg \omega_c$: quasi-static (flexibility “follows” the rigid motion)
- $\omega_k \approx \omega_c$: resonance risk (flexibility must be modeled dynamically)
- $\omega_k \ll \omega_c$: the system is effectively another rigid body (the flexible mode is so slow it acts like a separate DOF)

For the human body during locomotion, the long bones satisfy $\omega_k \gg \omega_c$ —bone compliance can be neglected. For the spine and for soft tissues, the separation is less clean, and dynamic flexibility effects may be significant.

1.2.2 Muscle Force Production Is State-Dependent

The most profound difference between engineered and biological actuators is that muscle force depends on the *state of the muscle itself*. An electric motor produces torque proportional to current, regardless of shaft angle or angular velocity (to first approximation). A muscle’s maximum force depends on:

1. **Muscle length** (force-length relationship): maximum isometric force occurs at an intermediate “optimal” length ℓ_0 , with force decreasing at shorter and longer lengths.
2. **Contraction velocity** (force-velocity relationship): concentric (shortening) contractions produce less force at higher velocities; eccentric (lengthening) contractions can produce forces exceeding the maximum isometric force by 50–80%.
3. **Activation level**: the fraction of motor units recruited and their firing rates, which follows its own first-order dynamics with time constants of 10–50 ms for activation and 40–200 ms for deactivation.

The combined force production model (Hill model, Chapter 3) is:

$$F_{\text{muscle}} = a(t) \cdot f_L(\tilde{\ell}) \cdot f_V(\tilde{v}) \cdot F_0 + f_{\text{PE}}(\tilde{\ell}), \quad (1.2)$$

where $a(t) \in [0, 1]$ is the activation level, f_L and f_V are the normalized force-length and force-velocity curves, F_0 is the maximum isometric force, $\tilde{\ell} = \ell/\ell_0$ and $\tilde{v} = v/v_0$ are normalized length and velocity, and f_{PE} is the passive elastic force.

Clinical Significance 1.1. Why This Matters for the Golf Swing

During the downswing, the wrist extensors are rapidly lengthening (eccentric contraction) while the shoulder muscles are shortening (concentric). The force capacity at

the wrist is therefore *enhanced* by the force-velocity relationship, while that at the shoulder is *diminished*. An engineering model that assigns constant torque limits to each joint completely misses this asymmetry, which is fundamental to the kinetic chain mechanism of the swing.

1.2.3 Redundancy Is a Feature, Not a Bug

The human body has approximately 630 skeletal muscles controlling approximately 240 joint degrees of freedom. Even for a simple task like reaching for a cup (which constrains only 6 degrees of freedom—3 position + 3 orientation of the hand), there are vastly more muscles and joints than constraints. This *motor redundancy* means that infinitely many muscle activation patterns can produce the same hand trajectory.

In engineering, redundancy is minimized to reduce cost. In biology, redundancy is *the solution to robustness*:

- If one muscle is fatigued, others can compensate.
- If a joint is injured, the task can be accomplished through alternative kinematic chains.
- Variability in execution—far from being noise to be eliminated—allows the system to explore the space of solutions and find robust attractors (see Volume IV for the motor control implications).

This connects directly to the *uncontrolled manifold hypothesis* [?]: task-irrelevant variability is not suppressed by the nervous system, only task-relevant variability is. The biological control system does not solve the full redundancy problem; it projects the problem onto the task-relevant subspace.

1.3 Modeling Strategies

Given the complexity of biological systems, modelers must choose their level of abstraction carefully. Three broad strategies exist, listed in order of increasing fidelity:

1.3.1 Strategy 1: Rigid-Body with Torque Actuators

The simplest approach: treat biological segments as rigid bodies and joints as ideal revolute, with “joint torques” as the control input. This is the model assumed throughout Volumes I–II.

Advantages:

- Mathematically tractable (standard Lagrangian mechanics)
- Sufficient for gross motion analysis
- Compatible with all control-theoretic tools from Volumes I–II

Limitations:

- Cannot predict muscle activation patterns
- Cannot account for muscle force-length-velocity constraints
- Cannot predict injury mechanisms
- Cannot capture bi-articular muscle effects

When to use: Trajectory optimization, gross motion planning, initial controller design.

1.3.2 Strategy 2: Rigid-Body with Muscle Actuators

Replace torque actuators with Hill-type muscle models (Chapter 3). Joint torques become *outputs* of the model, computed from muscle forces and moment arms:

$$\tau_j = \sum_{i \in \mathcal{M}_j} r_{ij}(\mathbf{q}) \cdot F_{\text{muscle},i}, \quad (1.3)$$

where $\mathcal{M}_j$ is the set of muscles crossing joint j and $r_{ij}(\mathbf{q})$ is the moment arm of muscle i about joint j .

Advantages:

- Predicts muscle activation patterns
- Captures force-length-velocity constraints
- Accounts for bi-articular coupling
- Can predict co-contraction and joint stiffness modulation

Limitations:

- Significant increase in model complexity (typically 2–3 states per muscle)
- Muscle parameters are difficult to measure in vivo
- Still assumes rigid segments and ideal joints

When to use: Detailed motion analysis, muscle force prediction, clinical biomechanics, prosthetic design.

1.3.3 Strategy 3: Flexible Multi-Body with Muscle Actuators

The most complete approach: flexible bodies (beam or FEM models) with muscle actuators and realistic joint models. This is the domain of software platforms like OpenSim and AnyBody.

Advantages:

- Highest fidelity
- Can predict stress distributions in bones and soft tissues
- Can capture coupled joint kinematics

Limitations:

- Computationally expensive
- Many parameters, often poorly known
- Too complex for real-time control applications

When to use: Injury prediction, surgical planning, detailed structural analysis.

```

1 import numpy as np
2
3 def select_modeling_strategy(
4     need_muscle_forces: bool,
5     need_tissue_stress: bool,
6     real_time_required: bool,
7     n_dof: int
8 ) -> str:
9     """Select the appropriate biomechanical modeling strategy.
10
11     Parameters
12     -----
13     need_muscle_forces : bool
14         Whether individual muscle forces are needed.
15     need_tissue_stress : bool
16         Whether stress/strain in tissues is needed.
17     real_time_required : bool
18         Whether the model must run in real-time.
19     n_dof : int
20         Number of degrees of freedom in the model.
21
22     Returns
23     -----
24     str
25         Recommended modeling strategy.
26     """
27     if need_tissue_stress:
28         return "Strategy 3: Flexible multi-body + muscles (OpenSim/FEM)"
29     elif need_muscle_forces:
30         if real_time_required and n_dof > 20:
31             return ("Strategy 2 (reduced): Muscle model with "
32                     "synergy-based dimensionality reduction")
33         return "Strategy 2: Rigid-body + Hill-type muscles (OpenSim)"
34     else:
35         if real_time_required:
36             return "Strategy 1: Rigid-body + torque actuators (fastest)"
37         return ("Strategy 1 or 2 depending on available data "
38                 "and analysis goals")

```

Listing 1.1: Choosing a modeling strategy based on application requirements.

1.4 A Quantitative Tour of the Human Body

Before diving into the detailed modeling chapters, we present a quantitative overview of the key biomechanical properties that any model must capture.

1.4.1 Segment Inertial Properties

The human body can be decomposed into 15–20 segments for modeling purposes. The standard segmentation and mass fractions are based on cadaver studies, most notably those of Dempster (1955), updated by de Leva (1996) [?].

Segment	Mass (% body)	Length (cm, 180cm male)	CoM (% proximal)
Head + neck	8.1	25.4	50.0
Trunk	49.7	53.0	44.9
Upper arm	2.7	28.2	57.7
Forearm	1.6	26.9	45.7
Hand	0.6	19.0	36.2
Thigh	14.2	42.2	40.9
Shank	4.3	43.4	43.4
Foot	1.4	25.8	44.2

```

1 import numpy as np
2 from dataclasses import dataclass
3
4 @dataclass
5 class SegmentProperties:
6     """Inertial properties of a body segment."""
7     name: str
8     mass: float          # kg
9     length: float        # m
10    com_proximal: float   # fraction from proximal end
11    I_com: float          # moment of inertia about CoM, kg*m^2
12
13 def de_leva_male_segments(
14     body_mass: float,
15     body_height: float
16 ) -> list[SegmentProperties]:
17     """Compute segment properties using de Leva (1996) regression.
18
19     Parameters
20     -----
21     body_mass : float
22         Total body mass in kg.
23     body_height : float
24         Total body height in m.
25
26     Returns
27     -----
28     list[SegmentProperties]
29         List of segment inertial properties.
30
31     References
32     -----
33     de Leva, P. (1996). Adjustments to Zatsiorsky-Seluyanov's
34     segment inertia parameters. J. Biomechanics, 29(9), 1223-1230.

```

```

35     """
36     # Mass fractions, length fractions, CoM fractions, RoG fractions
37     # (proximal reference, male data from de Leva 1996 Table 4)
38     data = {
39         'head_neck': (0.0810, 0.1414, 0.5002, 0.3030),
40         'trunk': (0.4970, 0.2946, 0.4486, 0.3271),
41         'upper_arm': (0.0271, 0.1566, 0.5772, 0.2850),
42         'forearm': (0.0162, 0.1494, 0.4574, 0.2760),
43         'hand': (0.0061, 0.1057, 0.3624, 0.2880),
44         'thigh': (0.1416, 0.2321, 0.4095, 0.3290),
45         'shank': (0.0433, 0.2411, 0.4340, 0.2550),
46         'foot': (0.0137, 0.1527, 0.4415, 0.2570),
47     }
48
49     segments = []
50     for name, (mass_frac, len_frac, com_frac, rog_frac) in data.items():
51         seg_mass = mass_frac * body_mass
52         seg_length = len_frac * body_height
53         seg_com = com_frac
54         # Moment of inertia:  $I = m * (rog * length)^2$ 
55         seg_I = seg_mass * (rog_frac * seg_length) ** 2
56         segments.append(SegmentProperties(
57             name=name, mass=seg_mass, length=seg_length,
58             com_proximal=seg_com, I_com=seg_I
59         ))
60
61     return segments
62
63
64 # Example usage
65 if __name__ == "__main__":
66     segments = de_leva_male_segments(body_mass=80.0, body_height=1.80)
67     print(f"{'Segment':<15} {'Mass (kg)':>10} {'Length (m)':>10} "
68           f"{'I_com (kg*m^2)':>14}")
69     print("-" * 52)
70     for seg in segments:
71         print(f"{'seg.name':<15} {'seg.mass:>10.2f} {'seg.length:>10.3f} "
72               f"{'seg.I_com:>14.4f}")

```

Listing 1.2: Computing segment inertial properties from body measurements.

1.4.2 Joint Ranges of Motion

Each joint has characteristic ranges of motion that define the configuration space $\mathcal{Q}$ of the body. Unlike robot joints, which can be modeled as $q_i \in [q_i, \bar{q}_i]$, biological joints have *soft limits*: resistance increases gradually as the joint approaches its anatomical limit, governed by the elasticity of ligaments and joint capsule.

1.4.3 Muscle Properties

The human body contains approximately 630 muscles, each characterized by:

- Maximum isometric force F_0 (ranging from 10 N for small hand muscles to over 6000 N for the quadriceps)

- Optimal fiber length ℓ_0 (the length at which maximum isometric force is produced)
- Maximum contraction velocity v_0 (typically 5–15 fiber lengths per second)
- Tendon slack length ℓ_T^s (the length at which the tendon begins to bear load)
- Pennation angle α_0 (the angle between muscle fibers and the line of action, ranging from 0° for parallel-fibered muscles to 30° or more for pennate muscles like the quadriceps)

These properties have been systematically catalogued for major musculoskeletal models [?, ?].

1.5 Connecting to the Control Framework

Despite the added complexity, the control framework of Volumes I and II remains applicable—with modifications. The key insight is:

Biomechanical Principle 1.2. The Extended Affine Structure

A musculoskeletal system can be written in control-affine form as

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{G}(\mathbf{x})\mathbf{u}, \quad (1.4)$$

where the state $\mathbf{x}$ now includes muscle states (activation, fiber length) in addition to joint coordinates and velocities, the drift field $\mathbf{f}$ includes passive muscle and tendon forces, and $\mathbf{G}(\mathbf{x})$ maps neural excitation signals $\mathbf{u}$ to state derivatives through the muscle activation and contraction dynamics. The dimensions expand—from $2n$ states with n joints and m torque actuators to $2n+2p$ states with p muscles—but the *structure* of the problem (drift + control) is preserved.

In Plain Language 1.1. Mathematical Reminder: The Expanded Manifold

Recall from Volumes 0 and I that the "State Vector" $\mathbf{x}$ is a point moving through a geometric space called the State Space Manifold.

In robotics, this manifold just tracked joint angles and velocities ($2n$ dimensions). In biomechanics, we are mathematically expanding the dimensional size of the manifold by adding $2p$ brand new dimensions to track muscle activation and fiber length. The system is still just a single dot moving through a manifold governed by $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{G}(\mathbf{x})\mathbf{u}$, but the geometry is now vastly higher-dimensional. All of the differential geometry tools (Tangent Spaces, Logarithms) still mechanically apply, but at a computational cost!

This means:

- The tangent-space analysis of Volume I applies to the expanded state space
- Contraction theory (Volume I, Chapter 4) can assess stability of the muscle-driven system

- Trajectory optimization (Volume I, Chapter 5) extends to find optimal neural excitation patterns
- The counterfactual analysis (Volume I, Chapter 7) can distinguish which aspects of muscle force production contribute to the motion

The challenge is *dimensionality*: a 20-DOF body with 100 muscles has a state space of dimension $2 \times 20 + 2 \times 100 = 240$. This is precisely the “curse of dimensionality” that Volume IV addresses through the lens of motor control and neural architecture.

Exercises

1. **Segment Properties.** Using the `de.leiva.male.segments()` function, compute the total mass and center of mass of a 75-kg, 1.75-m male in the anatomical position. Verify that the total mass sums to the body mass.
2. **Compliance Effects.** A golf club shaft has a first bending mode at approximately 5 Hz. If the downswing takes 0.25 s, how many oscillation cycles of the shaft occur during the downswing? Does the timescale separation principle (Principle 1.1) justify neglecting shaft flexibility? Discuss.
3. **Redundancy Quantification.** A planar model of the arm has 3 joints (shoulder, elbow, wrist) and 6 muscles. If the task is to place the fingertip at a given (x, y) position, what is the dimension of the null space? How many independent muscle patterns produce the same fingertip position?
4. **Force-Velocity Impact.** During a golf downswing, the wrist extensors lengthen at approximately 5 fiber lengths per second. Using the Hill force-velocity relationship (Equation 1.2), estimate the ratio of eccentric force to maximum isometric force at this velocity. Compare this with the force available during a slow concentric contraction.
5. **Modeling Strategy Selection.** For each of the following applications, select the appropriate modeling strategy and justify your choice:
 - (a) Real-time control of a prosthetic knee during walking
 - (b) Predicting ACL stress during a cutting maneuver in soccer
 - (c) Optimizing a sprinter’s start to minimize 10-m time
 - (d) Understanding how the golf swing generates clubhead speed
6. **(Programming.)** Implement a function that computes the total gravitational potential energy of a multi-segment body given segment properties and joint angles. Test it for a 2-segment (upper arm + forearm) model in different configurations.
7. **(Programming.)** Extend the segment properties code to compute the full 6×6 spatial inertia matrix (in the Park and Lynch spatial algebra notation) for each segment. Verify that the matrices are positive definite.

8. **(Research.)** Compare the de Leva (1996) segment parameters with at least one other source (e.g., Winter 2009, Dempster 1955). Quantify the differences in mass fractions and discuss the implications for inverse dynamics calculations.

Chapter 2

Musculoskeletal Modeling Conventions

In biomechanics, the first source of error is not the mathematics but the coordinate system.

— ISB Standardization Committee

2.1 Anatomical Reference Frames

Biomechanical analysis requires a standardized language for describing body position and motion. The International Society of Biomechanics (ISB) has published recommendations for joint coordinate systems [?, ?] that serve as the international standard.

2.1.1 The Anatomical Position

All references to body position begin from the **anatomical position**: the subject stands upright, feet together, arms at sides, palms facing forward. This defines the *zero configuration* from which all joint angles are measured.

Definition 2.1. Global Reference Frame

The global (laboratory) reference frame $\{G\}$ is defined as:

- $\hat{x}_G$: anterior (forward)
- $\hat{y}_G$: superior (upward)
- $\hat{z}_G$: lateral right

This is a right-handed coordinate system. Note that different communities use different conventions: biomechanics typically uses *Y*-up, while robotics often uses *Z*-up. **Always verify the convention before combining data from different sources.**

2.1.2 Segment Reference Frames

Each body segment has a local reference frame attached to it, defined by bony landmarks that can be palpated or identified from medical imaging. The ISB recommendations specify these landmarks for each segment.

Example 2.1. The Thigh Reference Frame

The ISB-recommended thigh reference frame $\{T\}$ is defined by:

- **Origin:** midpoint between medial and lateral femoral epicondyles
- $\hat{\mathbf{y}}_T$: from origin to hip joint center (proximal direction)
- $\hat{\mathbf{z}}_T$: perpendicular to the plane formed by the two epicondyles and the hip center, pointing laterally
- $\hat{\mathbf{x}}_T$: cross product $\hat{\mathbf{y}}_T \times \hat{\mathbf{z}}_T$ (anterior direction)

The hip joint center is estimated from the pelvis landmarks using regression equations (e.g., Harrington et al. 2007).

2.1.3 Connection to Screw Theory

In the language of Volume 0, each segment frame $\{S_i\}$ is related to the global frame $\{G\}$ by a homogeneous transformation matrix $T_{GS_i} \in \text{SE}(3)$:

$$T_{GS_i} = \begin{bmatrix} R_{GS_i} & \mathbf{p}_{GS_i} \\ \mathbf{0}^T & 1 \end{bmatrix}, \quad (2.1)$$

where $R_{GS_i} \in \text{SO}(3)$ is the rotation matrix and $\mathbf{p}_{GS_i} \in \mathbb{R}^3$ is the position of the segment origin in the global frame.

The *relative configuration* of segment j with respect to proximal segment i is:

$$T_{S_i S_j} = T_{GS_i}^{-1} T_{GS_j}. \quad (2.2)$$

Joint angles are extracted from this relative transformation.

2.2 Joint Coordinate Systems

The ISB recommends decomposing the relative rotation $R_{S_i S_j}$ into three angles using a specific Euler angle sequence for each joint. The choice of sequence is *not arbitrary*—it is selected to align with the clinical definitions of flexion-extension, abduction-adduction, and internal-external rotation.

2.2.1 The Grood–Suntay Convention for the Knee

The knee joint coordinate system, originally proposed by Grood and Suntay (1983) [?], decomposes knee motion into:

1. **Flexion-Extension** (α): rotation about the femoral medio-lateral axis (the “fixed” axis of the femur)
2. **Abduction-Adduction** (β): rotation about the “floating” axis (perpendicular to both the femoral and tibial body-fixed axes)

3. Internal-External Rotation (γ): rotation about the tibial longitudinal axis (the “body-fixed” axis of the tibia)

This is a ZXY Euler angle sequence when expressed in the Grood–Suntay convention. The key insight is that the floating axis is neither fixed in the femur nor in the tibia—it is defined by the cross product of the other two axes.

```

1 import numpy as np
2 from typing import Tuple
3
4 def grood_suntay_knee_angles(
5     R_femur: np.ndarray,
6     R_tibia: np.ndarray
7 ) -> Tuple[float, float, float]:
8     """Compute knee joint angles using Grood-Suntay convention.
9
10    Parameters
11    -----
12    R_femur : np.ndarray, shape (3, 3)
13        Rotation matrix of femur segment frame in global coordinates.
14    R_tibia : np.ndarray, shape (3, 3)
15        Rotation matrix of tibia segment frame in global coordinates.
16
17    Returns
18    -----
19    Tuple[float, float, float]
20        (flexion_deg, adduction_deg, internal_rotation_deg)
21
22    References
23    -----
24    Grood, E.S. and Suntay, W.J. (1983). A joint coordinate system
25    for the clinical description of three-dimensional motions.
26    J. Biomechanical Engineering, 105(2), 136-144.
27    """
28    # Femoral medio-lateral axis (e1) - flexion axis
29    e1 = R_femur[:, 2] # lateral axis of femur
30
31    # Tibial longitudinal axis (e3) - rotation axis
32    e3 = R_tibia[:, 1] # proximal axis of tibia
33
34    # Floating axis (e2) - perpendicular to both
35    e2 = np.cross(e3, e1)
36    e2 = e2 / np.linalg.norm(e2)
37
38    # Flexion-extension angle
39    # Angle between femur anterior axis and floating axis
40    femur_anterior = R_femur[:, 0]
41    flexion = np.arctan2(
42        np.dot(e2, R_femur[:, 1]),
43        np.dot(e2, femur_anterior)
44    )
45
46    # Abduction-adduction angle
47    # Angle between e1 and e3 minus 90 degrees
48    sin_abd = -np.dot(e1, e3)
49    cos_abd = np.linalg.norm(np.cross(e1, e3))

```

```

50     adduction = np.arcsin(np.clip(sin_abd, -1.0, 1.0))
51
52     # Internal-external rotation
53     # Angle between floating axis and tibia anterior axis
54     tibia_anterior = R_tibia[:, 0]
55     internal_rot = np.arctan2(
56         np.dot(e2, R_tibia[:, 2]),
57         np.dot(e2, tibia_anterior)
58     )
59
60     return (
61         np.degrees(flexion),
62         np.degrees(adduction),
63         np.degrees(internal_rot)
64     )

```

Listing 2.1: Computing knee joint angles using the Grood-Suntay convention.

2.2.2 Multi-Representation Joint Angles

Following the multi-representation philosophy of this series, we express joint rotations in all four standard representations:

1. Euler Angles (ISB Convention):

$$R_{ij} = R_z(\alpha) R_x(\beta) R_y(\gamma), \quad (2.3)$$

where α, β, γ are the clinical angles (flexion, adduction, rotation).

2. Rotation Matrix (SO(3)):

$$R_{ij} = \exp([\omega]_{\times} \theta) \in \text{SO}(3), \quad (2.4)$$

which avoids gimbal lock but requires 9 parameters with 6 constraints.

3. Unit Quaternion:

$$\mathbf{q} = (\cos \frac{\theta}{2}, \boldsymbol{\omega} \sin \frac{\theta}{2}), \quad \|\mathbf{q}\| = 1, \quad (2.5)$$

which is singularity-free and computationally efficient.

4. Screw Axis (Primary):

$$T_{ij} = e^{[S]\theta} \in \text{SE}(3), \quad \mathcal{S} = (\boldsymbol{\omega}, \mathbf{v}) \in \mathbb{R}^6, \quad (2.6)$$

which captures the coupled rotation-translation of biological joints naturally.

Biology vs. Engineering 2.1. Why Screw Axis Theory Is Natural for Biological Joints

Unlike engineering revolute joints (pure rotation about a fixed axis), biological joints exhibit *coupled* rotation and translation. The knee slides anteriorly as it flexes; the glenohumeral joint translates inferiorly during abduction. Screw axis theory captures this coupling naturally: every rigid-body motion is a rotation about and translation along a screw axis. The instantaneous screw axis (ISA) of a biological joint moves

through space as the joint moves, unlike the fixed axis of an ideal revolute. This is precisely why Park and Lynch's *Modern Robotics* framework is preferred: it handles coupled rotation-translation without special cases.

2.3 Body Segment Parameters

2.3.1 Regression Equations

The most widely used body segment parameter (BSP) data comes from:

1. **Dempster (1955)**: cadaver study of 8 elderly males. Still widely cited but limited demographic range.
2. **Zatsiorsky & Seluyanov (1983, 1990)**: gamma-ray scanning of 100 living young males. More representative but uses different segment definitions.
3. **de Leva (1996)**: adjustment of Zatsiorsky data to standard segment definitions. The current gold standard for adult male and female BSP.
4. **Pavol et al. (2002)**: elderly adult data.
5. **Jensen (1986, 1989)**: pediatric data.

2.3.2 Computing the Mass Matrix

Given segment inertial properties, the mass matrix $M(\mathbf{q}) \in \mathbb{R}^{n \times n}$ of the multi-body system is computed using the composite rigid-body algorithm (Volume 0, Chapter 10). In the body-fixed frame of segment i , the spatial inertia is:

$$\mathcal{G}_i = \begin{bmatrix} I_i & m_i[\mathbf{c}_i]_{\times}^T \\ m_i[\mathbf{c}_i]_{\times} & m_i\mathbb{I}_3 \end{bmatrix} \in \mathbb{R}^{6 \times 6}, \quad (2.7)$$

where I_i is the 3×3 rotational inertia about the segment frame origin, m_i is the segment mass, $\mathbf{c}_i$ is the center of mass position in the segment frame, and $[\cdot]_{\times}$ denotes the skew-symmetric matrix.

Exercises

1. **Coordinate System Verification.** Verify that the Grood–Suntay knee joint coordinate system produces zero angles when the femur and tibia reference frames are aligned. Test with known rotations.
2. **Euler Angle Singularity.** For the ZXY Euler angle sequence used in the knee, identify the gimbal lock configuration. Is this configuration ever reached during normal gait? During deep squatting?

3. **Multi-Representation Conversion.** Write Python functions to convert between all four representations (Euler ZXY, rotation matrix, quaternion, screw axis) for a knee joint angle of 60° flexion, 5° adduction, 10° internal rotation.
4. **BSP Sensitivity.** Using the de Leva segment properties, compute the knee extension torque required to hold the lower leg horizontal (isometric). Repeat using the Dempster data. What is the percentage difference? Discuss the implications for inverse dynamics accuracy.
5. **(Programming.)** Implement the spatial inertia matrix computation for a 3-segment leg model (thigh, shank, foot). Verify positive-definiteness.
6. **(Programming.)** Using motion capture data (or simulated data), compute knee joint angles from segment orientations using the Grood–Suntay code provided. Plot flexion angle over one gait cycle.

Chapter 3

Muscle Models

A muscle is not a motor. It is a spring with a memory, a damper with an agenda, and a force generator with a personality.

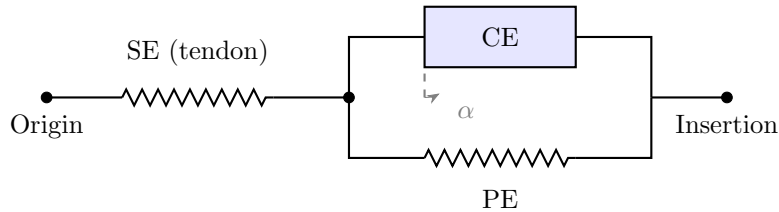
3.1 The Hill-Type Muscle Model

The Hill-type muscle model, originating from A.V. Hill's Nobel Prize-winning work on muscle thermodynamics [?], is the standard lumped-parameter model used in biomechanical simulation. It represents the musculotendon unit as a combination of contractile, elastic, and damping elements.

3.1.1 Model Architecture

The standard Hill-type musculotendon model consists of three elements:

1. **Contractile Element (CE)**: represents the active force-generating capacity of the muscle fibers. Its force depends on activation $a(t)$, fiber length ℓ^M , and fiber velocity $v^M = \dot{\ell}^M$.
2. **Parallel Elastic Element (PE)**: represents the passive elastic properties of the muscle belly (titin filaments, connective tissue). Produces force only when the muscle is stretched beyond its resting length.
3. **Series Elastic Element (SE)**: represents the tendon, which is in series with the contractile element. Modeled as a nonlinear spring with a slack length ℓ_T^s .



3.1.2 The Force-Length Relationship

The isometric (zero velocity) force of the contractile element depends on the normalized fiber length $\tilde{\ell} = \ell^M / \ell_0^M$:

Definition 3.1. Active Force-Length Curve

The normalized active force-length relationship is:

$$f_L(\tilde{\ell}) = \exp\left(-\frac{(\tilde{\ell} - 1)^2}{\gamma}\right), \quad (3.1)$$

where $\gamma \approx 0.45$ controls the width of the curve. Maximum force ($f_L = 1$) occurs at optimal fiber length ($\tilde{\ell} = 1$). Force decreases for both shorter and longer fiber lengths, reaching near-zero at approximately $\tilde{\ell} = 0.5$ and $\tilde{\ell} = 1.5$.

The physical basis for the force-length relationship is the sliding filament theory [?]: at the optimal length, the maximum number of actin-myosin cross-bridges can form. At shorter lengths, filament overlap reduces the number of available binding sites. At longer lengths, the filaments pull apart.

```

1 import numpy as np
2 from dataclasses import dataclass
3 from typing import Tuple
4
5 @dataclass
6 class MuscleParameters:
7     """Parameters for a Hill-type muscletendon model."""
8     F0: float          # Maximum isometric force (N)
9     l0_M: float         # Optimal fiber length (m)
10    v0: float           # Maximum contraction velocity (m/s)
11    l_T_slack: float     # Tendon slack length (m)
12    alpha0: float        # Pennation angle at optimal length (rad)
13    gamma: float = 0.45  # Force-length curve width parameter
14    Af: float = 0.25     # Force-velocity shape factor
15    kPE: float = 4.0     # Passive elastic stiffness parameter
16    e0: float = 0.6      # Passive elastic strain at F0
17
18    @property
19    def v_max(self) -> float:
20        """Maximum shortening velocity in fiber lengths/s."""
21        return self.v0 / self.l0_M
22
23
24 def force_length_active(l_tilde: float, gamma: float = 0.45) -> float:
25     """Active force-length relationship.
26
27     Parameters
28     -----
29     l_tilde : float
30         Normalized fiber length (l_M / l0_M).
31     gamma : float
32         Width parameter of the Gaussian curve.
33
34     Returns
35     -----
36     float
37         Normalized active force (0 to 1).
38

```

```

39     References
40     -----
41     Thelen, D.G. (2003). Adjustment of muscle mechanics model
42     parameters to simulate dynamic contractions in older adults.
43     J. Biomechanical Engineering, 125(1), 70-77.
44     """
45     return np.exp(-((l_tilde - 1.0) ** 2) / gamma)
46
47
48 def force_length_passive(l_tilde: float, kPE: float = 4.0,
49                          e0: float = 0.6) -> float:
50     """Passive force-length relationship (exponential model).
51
52     Parameters
53     -----
54     l_tilde : float
55         Normalized fiber length.
56     kPE : float
57         Stiffness parameter.
58     e0 : float
59         Strain at which passive force equals F0.
60
61     Returns
62     -----
63     float
64         Normalized passive force.
65     """
66     if l_tilde <= 1.0:
67         return 0.0
68     return (np.exp(kPE * (l_tilde - 1.0) / e0 - kPE) - 1.0) / \
69            (np.exp(kPE) - 1.0)
70
71
72 def force_velocity(v_tilde: float, a: float, Af: float = 0.25,
73                   Flen: float = 1.4) -> float:
74     """Force-velocity relationship (Thelen 2003 model).
75
76     Parameters
77     -----
78     v_tilde : float
79         Normalized fiber velocity (v_M / v_max).
80         Negative = shortening (concentric).
81     a : float
82         Activation level (0 to 1).
83     Af : float
84         Shape factor for concentric portion.
85     Flen : float
86         Maximum normalized eccentric force.
87
88     Returns
89     -----
90     float
91         Normalized force multiplier.
92     """
93     if v_tilde <= 0:
94         # Concentric (shortening)

```

```

95         return (1.0 + v_tilde) / (1.0 - v_tilde / (Af * a + 0.01))
96     else:
97         # Eccentric (lengthening)
98         return (1.0 + v_tilde * Flen / (0.04 + a)) / \
99             (1.0 + v_tilde / (0.04 + a))
100
101
102 def muscle_force(
103     activation: float,
104     l_M: float,
105     v_M: float,
106     params: MuscleParameters
107 ) -> Tuple[float, float, float]:
108     """Compute total musculotendon force.
109
110     Parameters
111     -----
112     activation : float
113         Neural activation level (0 to 1).
114     l_M : float
115         Current muscle fiber length (m).
116     v_M : float
117         Current muscle fiber velocity (m/s, negative = shortening).
118     params : MuscleParameters
119         Muscle parameters.
120
121     Returns
122     -----
123     Tuple[float, float, float]
124         (total_force, active_force, passive_force) in Newtons.
125     """
126     # Normalize
127     l_tilde = l_M / params.l0_M
128     v_tilde = v_M / params.v0
129
130     # Force components
131     f_active = (activation
132                 * force_length_active(l_tilde, params.gamma)
133                 * force_velocity(v_tilde, activation, params.Af))
134     f_passive = force_length_passive(l_tilde, params.kPE, params.e0)
135
136     # Pennation angle (changes with fiber length)
137     cos_alpha = np.cos(params.alpha0) if l_tilde > 0.1 else 1.0
138
139     # Total force along tendon
140     F_active = f_active * params.F0 * cos_alpha
141     F_passive = f_passive * params.F0 * cos_alpha
142     F_total = F_active + F_passive
143
144     return F_total, F_active, F_passive
145
146
147 # Example: Biceps brachii
148 biceps = MuscleParameters(
149     F0=624.3,          # N (Arnold et al. 2010)
150     l0_M=0.1157,      # m

```

```

151     v0=10 * 0.1157, # 10 fiber lengths/s
152     l_T_slack=0.2723, # m
153     alpha0=0.0 # parallel-fibered
154 )
155
156 # Compute force at optimal length, half activation, isometric
157 F_total, F_active, F_passive = muscle_force(
158     activation=0.5,
159     l_M=biceps.l0_M, # optimal length
160     v_M=0.0, # isometric
161     params=biceps
162 )
163 print(f"Biceps at 50% activation, optimal length, isometric:")
164 print(f"    Active force: {F_active:.1f} N")
165 print(f"    Passive force: {F_passive:.1f} N")
166 print(f"    Total force: {F_total:.1f} N")

```

Listing 3.1: Hill-type muscle model implementation.

3.1.3 The Force-Velocity Relationship

The second critical property of the contractile element is the force-velocity relationship, first characterized by Hill (1938) [?].

Biomechanical Principle 3.1. The Force-Velocity Asymmetry

- **Concentric (shortening) contraction:** force *decreases* with increasing shortening velocity. At maximum shortening velocity v_0 , the muscle produces zero force. The relationship is approximately hyperbolic: $(F + a)(v + b) = (F_0 + a)b$.
- **Eccentric (lengthening) contraction:** force *increases* beyond the maximum isometric force, typically reaching $1.4\text{--}1.8 \times F_0$. This is why muscles are “stronger” when resisting a load than when lifting it.
- **Isometric** ($v = 0$): force equals the product of activation and the force-length relationship.

The asymmetry between concentric and eccentric force production is unique to biological actuators. No common engineering actuator exhibits this behavior.

3.1.4 Activation Dynamics

The neural command from the central nervous system does not instantly produce muscle force. The activation dynamics models the excitation-contraction coupling:

$$\dot{a}(t) = \frac{u(t) - a(t)}{\tau(u, a)}, \quad (3.2)$$

where $u(t) \in [0, 1]$ is the neural excitation, $a(t) \in [0, 1]$ is the activation level, and τ is a time constant that differs for activation and deactivation:

$$\tau(u, a) = \begin{cases} \tau_{\text{act}}(0.5 + 1.5a) & \text{if } u > a \text{ (activation),} \\ \tau_{\text{deact}}/(0.5 + 1.5a) & \text{if } u \leq a \text{ (deactivation).} \end{cases} \quad (3.3)$$

Typical values: $\tau_{\text{act}} = 10\text{--}15$ ms, $\tau_{\text{deact}} = 40\text{--}60$ ms [?]. The asymmetry between activation and deactivation time constants has important consequences for motor control: it is much easier to “turn on” a muscle quickly than to “turn it off.”

3.2 Tendon Mechanics

The tendon connects the muscle belly to bone. It is modeled as a nonlinear elastic element with a resting (slack) length ℓ_T^s :

$$F_T(\varepsilon_T) = \begin{cases} 0 & \varepsilon_T \leq 0, \\ F_0 \cdot \frac{e^{k_T \varepsilon_T} - 1}{e^{k_T \varepsilon_{0T}} - 1} & \varepsilon_T > 0, \end{cases} \quad (3.4)$$

where $\varepsilon_T = (\ell_T - \ell_T^s)/\ell_T^s$ is the tendon strain and $\varepsilon_{0T} \approx 0.033$ is the strain at maximum isometric force. The tendon stiffness parameter k_T is typically chosen so that the tendon strain is approximately 3.3% at F_0 [?].

Clinical Significance 3.1. Energy Storage in Tendons

Tendons are not merely passive connectors—they are *energy storage devices*. During running, the Achilles tendon stores approximately 35 J of elastic energy per stride, which is returned during push-off. This reduces the metabolic cost of running by approximately 50% compared to a hypothetical system with rigid tendons [?].

For the golf swing, the tendons of the wrist and forearm muscles store energy during the early downswing (when the wrist is cocked) and release it during the uncocking phase, contributing to clubhead speed.

3.3 Musculotendon Equilibrium

The muscle fiber and tendon must satisfy force equilibrium at all times. Assuming the muscle fiber acts at a pennation angle α to the tendon line of action:

$$F_T = (F_{\text{CE}} + F_{\text{PE}}) \cos \alpha, \quad (3.5)$$

where:

$$F_{\text{CE}} = a \cdot f_L(\tilde{\ell}^M) \cdot f_V(\tilde{v}^M) \cdot F_0, \quad (3.6)$$

$$F_{\text{PE}} = f_{\text{PE}}(\tilde{\ell}^M) \cdot F_0. \quad (3.7)$$

The musculotendon length ℓ^{MT} is the sum of tendon length and the projection of fiber length:

$$\ell^{MT} = \ell_T + \ell^M \cos \alpha. \quad (3.8)$$

Given a known musculotendon length (from the skeletal geometry) and activation, the equilibrium condition (3.5) and the constraint (3.8) determine the fiber length and velocity.

3.4 The Complete Hill Model as a Dynamical System

Combining all elements, the Hill muscle model can be expressed as a dynamical system with two state variables per muscle:

$$\dot{a} = \frac{u - a}{\tau(u, a)}, \quad (3.9)$$

$$\dot{\ell}^M = v^M(a, \ell^M, \ell^{MT}), \quad (3.10)$$

where v^M is obtained by inverting the force-velocity relationship at the equilibrium fiber force. The input is the neural excitation $u(t) \in [0, 1]$ and the “disturbance” is the musculotendon path length $\ell^{MT}(\mathbf{q})$, which depends on the joint configuration.

Biomechanical Principle 3.2. Muscle as a Nonlinear Control System

Each muscle is itself a nonlinear dynamical system with:

- **State:** $(a, \ell^M) \in [0, 1] \times \mathbb{R}_{>0}$
- **Input:** neural excitation $u \in [0, 1]$
- **Output:** tendon force F_T
- **Disturbance:** musculotendon path length $\ell^{MT}(\mathbf{q})$

The muscle transforms a scalar neural command into a scalar force, but the transformation is highly nonlinear, state-dependent, and history-dependent. The entire musculoskeletal system is a cascade: neural commands $\rightarrow$ muscle dynamics $\rightarrow$ tendon forces $\rightarrow$ joint torques $\rightarrow$ skeletal dynamics.

Exercises

1. **Force-Length Curve.** Plot the active force-length curve for $\gamma \in \{0.3, 0.45, 0.6\}$. For each, determine the “useful range” (the interval of fiber lengths where $f_L > 0.5$). How does the width parameter affect the operating range?
2. **Force-Velocity Curve.** Plot the complete force-velocity curve (both concentric and eccentric) for the biceps parameters given in the code listing. Mark the maximum eccentric force and the maximum shortening velocity.
3. **Activation Dynamics.** Simulate the activation dynamics for a square-wave neural excitation input ($u = 1$ for 200 ms, then $u = 0$). Plot the activation response with $\tau_{\text{act}} = 15$ ms and $\tau_{\text{deact}} = 50$ ms. How long does it take for activation to reach 95% of maximum? How long for deactivation?
4. **Tendon Compliance.** For the Achilles tendon (slack length $\ell_T^s = 0.25$ m, max force 4000 N), compute the tendon stretch at 50%, 75%, and 100% of maximum force. Does the tendon behave approximately linearly over this range?

5. **Isometric Force Surface.** Create a 3D surface plot of muscle force as a function of normalized fiber length ($\tilde{\ell}$) and activation (a) at zero velocity (isometric). Identify the point of maximum force production.
6. **(Programming.)** Implement a complete musculotendon simulation: given a prescribed musculotendon length trajectory $\ell^{MT}(t)$ and neural excitation $u(t)$, integrate the state equations (3.9) and (3.10) and plot the resulting tendon force $F_T(t)$.
7. **(Programming.)** Implement a two-muscle antagonist pair (e.g., biceps and triceps) acting on a single-DOF elbow joint. Given co-contraction ($u_{\text{biceps}} = u_{\text{triceps}} = 0.5$), compute the net joint torque and the joint stiffness. Verify that co-contraction increases stiffness without changing the equilibrium position.
8. **(Research.)** Compare the Thelen (2003) muscle model with the Millard et al. (2013) model used in OpenSim 4.x. What are the key differences in the force-velocity and passive force models? Which is more physiologically accurate?

Chapter 4

Joint Kinematics and Soft Tissue

The knee is not a hinge. The shoulder is not a ball-and-socket. These are useful fictions that biology tolerates but does not enforce.

4.1 Beyond Ideal Joints

Engineering joints are designed to constrain motion to specific degrees of freedom: a revolute joint permits one rotation, a prismatic joint permits one translation. Biological joints achieve constraint through a fundamentally different mechanism: the interaction of articular surfaces, ligaments, joint capsule, and muscle forces. This section develops the kinematics of biological joints using the screw theory framework of Volume 0, extended to handle the additional complexity.

4.1.1 The Instantaneous Screw Axis

Every rigid-body motion can be decomposed into a rotation about and translation along a **screw axis** (Chasles' theorem, Volume 0, Chapter 5). For an engineering revolute joint, this screw axis is fixed in both bodies. For a biological joint, the instantaneous screw axis (ISA) *moves* as the joint configuration changes.

Definition 4.1. Instantaneous Screw Axis of a Biological Joint

Given the relative twist $\mathcal{V}_{ij} = (\boldsymbol{\omega}, \mathbf{v}) \in \mathbb{R}^6$ between adjacent segments i and j , the instantaneous screw axis has:

- Direction: $\hat{\mathbf{s}} = \boldsymbol{\omega} / \|\boldsymbol{\omega}\|$
- Pitch: $h = \boldsymbol{\omega} \cdot \mathbf{v} / \|\boldsymbol{\omega}\|^2$
- Location: $\mathbf{q} = \boldsymbol{\omega} \times \mathbf{v} / \|\boldsymbol{\omega}\|^2$

For a pure rotation ($h = 0$), the ISA passes through $\mathbf{q}$ in direction $\hat{\mathbf{s}}$. Tracking how $\mathbf{q}$ and $\hat{\mathbf{s}}$ change with joint angle reveals the coupled kinematics of the biological joint.

4.1.2 The Knee: Rolling, Sliding, and Coupled Rotation

The tibiofemoral joint exemplifies the complexity of biological kinematics. During flexion from 0° to 90° :

1. The femoral condyles **roll** on the tibial plateaus (like a wheel on a road)
2. They simultaneously **slide** posteriorly (the contact point moves backward)
3. The tibia undergoes coupled **internal rotation** of approximately 15° (the “screw-home” mechanism)
4. The contact area shifts, changing the effective moment arm

This coupled motion means the knee ISA migrates superiorly and posteriorly during flexion. The four-bar linkage model [?] provides a simplified but insightful mechanical analog:

```

1 import numpy as np
2 from typing import Tuple
3
4 def compute_isa(
5     omega: np.ndarray,
6     v: np.ndarray
7 ) -> Tuple[np.ndarray, np.ndarray, float]:
8     """Compute the instantaneous screw axis from a twist.
9
10    Parameters
11    -----
12    omega : np.ndarray, shape (3,)
13        Angular velocity vector.
14    v : np.ndarray, shape (3,)
15        Linear velocity of the body-fixed frame origin.
16
17    Returns
18    -----
19    Tuple[np.ndarray, np.ndarray, float]
20        (point_on_axis, axis_direction, pitch)
21    """
22    omega_norm = np.linalg.norm(omega)
23    if omega_norm < 1e-10:
24        # Pure translation
25        direction = v / np.linalg.norm(v)
26        return np.zeros(3), direction, np.inf
27
28    direction = omega / omega_norm
29    pitch = np.dot(omega, v) / omega_norm**2
30    point = np.cross(omega, v) / omega_norm**2
31
32    return point, direction, pitch
33
34
35 def knee_isa_trajectory(
36     flexion_angles: np.ndarray,
37     roll_slide_ratio: float = 0.6
38 ) -> np.ndarray:
39     """Estimate knee ISA position during flexion.
```

```

40
41 Simple model: ISA migrates posteriorly and superiorly
42 with flexion, governed by the roll-to-slide ratio.
43
44 Parameters
45 -----
46 flexion_angles : np.ndarray
47     Array of flexion angles in degrees.
48 roll_slide_ratio : float
49     Ratio of rolling to total contact displacement.
50     Pure rolling = 1.0, pure sliding = 0.0.
51
52 Returns
53 -----
54 np.ndarray, shape (N, 3)
55     ISA positions in the tibial reference frame.
56 """
57 R_condyle = 0.030 # approximate femoral condyle radius (m)
58 isa_positions = np.zeros((len(flexion_angles), 3))
59
60 for i, theta in enumerate(np.radians(flexion_angles)):
61     # Posterior migration due to rolling
62     posterior = R_condyle * theta * roll_slide_ratio
63     # Superior migration (ISA moves up with flexion)
64     superior = R_condyle * (1.0 - np.cos(theta * 0.5))
65
66     isa_positions[i] = [-posterior, superior, 0.0]
67
68 return isa_positions

```

Listing 4.1: Instantaneous screw axis computation for a biological joint.

4.1.3 The Shoulder: The Most Complex Joint

The glenohumeral joint has the largest range of motion of any joint in the body, achieved at the expense of stability. The “ball-and-socket” description is a gross simplification: the humeral head translates 2–4 mm during abduction, and the instantaneous center of rotation shifts continuously.

The shoulder complex involves four joints:

1. **Glenohumeral** (GH): primary ball-and-socket, 3 DOF rotation + ~ 3 mm translation
2. **Acromioclavicular** (AC): 3 DOF rotation, small range
3. **Sternoclavicular** (SC): 3 DOF rotation, small range
4. **Scapulothoracic** (ST): not a true synovial joint; the scapula glides on the thorax, constrained as a *surface contact* rather than a pin joint

The scapulothoracic “joint” is particularly interesting from a mathematical perspective: it is a *holonomic constraint* that the scapula must remain on the thoracic surface, effectively acting as a 2-DOF constraint (the scapula can translate on the surface but cannot lift off).

4.2 Ligament Models

Ligaments constrain joint motion by resisting excessive displacement. They are modeled as nonlinear springs with:

$$F_{\text{lig}}(\varepsilon) = \begin{cases} 0 & \varepsilon \leq 0 \text{ (slack)}, \\ k_1 \varepsilon^2 / (4\varepsilon_1) & 0 < \varepsilon \leq 2\varepsilon_1 \text{ (toe region)}, \\ k_2(\varepsilon - \varepsilon_1) & \varepsilon > 2\varepsilon_1 \text{ (linear region)}, \end{cases} \quad (4.1)$$

where $\varepsilon = (\ell - \ell_0)/\ell_0$ is the strain, ℓ_0 is the reference length, $\varepsilon_1 \approx 0.03$ is the transition strain, and k_1, k_2 are stiffness parameters [?].

4.3 Spinal Mechanics

The spine presents a unique modeling challenge: 24 vertebrae connected by intervertebral discs, each joint having 6 coupled degrees of freedom. The functional spinal unit (FSU) consisting of two adjacent vertebrae and their connecting disc is the basic mechanical unit.

The intervertebral disc consists of:

- **Nucleus pulposus:** a gel-like center that distributes compressive loads
- **Annulus fibrosus:** concentric rings of collagen fibers that resist torsion and bending

A common simplified model treats each FSU as a 6-DOF spring:

$$\mathbf{F}_{\text{disc}} = \mathbf{K}_{\text{disc}} \cdot \boldsymbol{\delta}, \quad (4.2)$$

where $\mathbf{K}_{\text{disc}} \in \mathbb{R}^{6 \times 6}$ is the stiffness matrix and $\boldsymbol{\delta} \in \mathbb{R}^6$ is the generalized displacement (3 translations + 3 rotations).

Exercises

1. **ISA Computation.** For a knee flexing at $60^\circ/\text{s}$ with 2 mm/s of posterior translation, compute the ISA direction, pitch, and location.
2. **Screw-Home Mechanism.** Plot the coupled internal rotation of the tibia as a function of knee flexion using the model: $\gamma = 15^\circ \cdot (1 - \cos(\theta/2))$ for $\theta \in [0, 90^\circ]$.
3. **Shoulder Kinematics.** Using screw axis theory, express the configuration of the hand (end-effector) as a product of exponentials through the SC, AC, GH, and elbow joints.
4. **(Programming.)** Implement the ligament force model and plot the force-strain curve. Identify the toe region and the linear region.
5. **(Programming.)** Create a 3D visualization of the knee ISA trajectory during flexion from 0° to 120° .

Chapter 5

Multi-Body Dynamics of Biological Chains

The human body is the most complex open kinematic chain in nature. Analyzing it with the tools of rigid-body dynamics is both necessary and insufficient.

5.1 Open-Chain vs. Closed-Chain Biomechanics

Most human movement involves open kinematic chains (the limbs) connected to a floating base (the pelvis/trunk). However, many functional activities create temporary closed chains: the stance phase of gait (foot on ground), pushing against a wall, or the golf swing when both hands grip the club.

5.1.1 Equations of Motion for Biological Chains

The equations of motion for an n -DOF musculoskeletal system take the standard Lagrangian form:

$$M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + g(\mathbf{q}) = R(\mathbf{q})\mathbf{F}_{\text{muscle}} + J_c^T(\mathbf{q})\mathbf{F}_{\text{contact}}, \quad (5.1)$$

where:

- $M(\mathbf{q}) \in \mathbb{R}^{n \times n}$ is the mass matrix (from segment inertial properties)
- $C(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^{n \times n}$ is the Coriolis/centrifugal matrix
- $g(\mathbf{q}) \in \mathbb{R}^n$ is the gravity vector
- $R(\mathbf{q}) \in \mathbb{R}^{n \times p}$ is the muscle moment arm matrix (p muscles, n DOF)
- $\mathbf{F}_{\text{muscle}} \in \mathbb{R}^p$ are the muscle forces (from Hill models, Chapter 3)
- $J_c(\mathbf{q})$ is the contact Jacobian
- $\mathbf{F}_{\text{contact}}$ are external contact forces

The key difference from Volume I's $\boldsymbol{\tau} = B\mathbf{u}$ is that joint torques are no longer direct control inputs. Instead:

$$\boldsymbol{\tau}_{\text{net}} = R(\mathbf{q})\mathbf{F}_{\text{muscle}} = \sum_{i=1}^p \mathbf{r}_i(\mathbf{q}) \cdot F_{\text{muscle},i}, \quad (5.2)$$

where $\mathbf{r}_i(\mathbf{q}) \in \mathbb{R}^n$ is the i -th column of R , giving the moment arm vector of muscle i across all joints.

5.1.2 Bi-articular Muscles

Unlike engineering actuators that span a single joint, many muscles cross two or more joints. These **bi-articular** (or multi-articular) muscles create mechanical coupling between joints that has no analog in standard robotics.

Biomechanical Principle 5.1. The Coordination Function of Bi-articular Muscles

Bi-articular muscles serve two distinct functions:

1. **Energy transfer:** they transport mechanical energy from one joint to another without metabolic cost. When the proximal joint extends (muscle shortening at proximal end) and the distal joint flexes (muscle lengthening at distal end), energy is transferred proximally to distally.
2. **Coordination constraint:** they link joint motions, reducing the effective degrees of freedom and simplifying the control problem.

This is a form of *mechanical intelligence*—the morphology of the musculoskeletal system performs part of the control task. The gastrocnemius, for example, crosses both the knee and ankle, coupling knee extension to ankle plantarflexion during the push-off phase of gait.

Control-Theoretic Perspective: In the language of Agrachev & Sachkov's geometric control theory [1], the moment arm matrix $R(\mathbf{q})$ is a covector field describing how muscle forces map to joint torques. Bi-articular muscles induce dependencies in the columns of $R(\mathbf{q})$, effectively constraining the input distribution $G(\mathbf{x})$ of the system. This reduces the effective control dimension and is formalized in Chapter 3 of [1] (reachability and controllability of affine systems).

```

1 import numpy as np
2
3 def compute_joint_torques(
4     moment_arms: np.ndarray,
5     muscle_forces: np.ndarray
6 ) -> np.ndarray:
7     """Compute net joint torques from muscle forces and moment arms.
8
9     Parameters
10    -----
11    moment_arms : np.ndarray, shape (n_joints, n_muscles)

```

```

12     Moment arm matrix R(q). Each column gives the moment arm
13     of one muscle across all joints. Sign convention: positive
14     moment arm produces positive joint torque (typically flexion).
15     muscle_forces : np.ndarray, shape (n_muscles,)
16     Force produced by each muscle (always positive).
17
18     Returns
19     -----
20     np.ndarray, shape (n_joints,)
21     Net joint torques.
22     """
23     return moment_arms @ muscle_forces
24
25
26 # Example: 2-DOF elbow system with 4 muscles
27 # Joints: shoulder flexion, elbow flexion
28 # Muscles: anterior deltoid, biceps (biarticular),
29 #          triceps (biarticular), brachialis
30 R = np.array([
31     # shoulder, elbow
32     [0.05, 0.03, -0.025, 0.0], # shoulder moment arms (m)
33     [0.0, 0.04, -0.020, 0.03], # elbow moment arms (m)
34 ]).T # Shape: (4 muscles, 2 joints) -> transpose to (2, 4)
35 R = R.T # Now (2 joints, 4 muscles)
36
37 # Note: biceps has moment arms at BOTH joints (bi-articular)
38 muscle_forces = np.array([200.0, 150.0, 100.0, 120.0]) # Newtons
39
40 tau = compute_joint_torques(R, muscle_forces)
41 print(f"Shoulder torque: {tau[0]:.1f} N*m")
42 print(f"Elbow torque: {tau[1]:.1f} N*m")

```

Listing 5.1: Computing joint torques from muscle forces through moment arms.

5.1.3 Co-Contraction and Joint Stiffness

When antagonist muscles co-contract (both agonist and antagonist are active simultaneously), the net joint torque may be zero but the joint **stiffness** increases. This is because each active muscle acts as a spring:

$$K_{\text{joint}} = \sum_{i=1}^p r_i^2(\mathbf{q}) \cdot \frac{\partial F_{\text{muscle},i}}{\partial \ell_i^M}, \quad (5.3)$$

where $\partial F / \partial \ell^M$ is the short-range stiffness of the muscle, which depends on activation level.

Co-contraction is metabolically expensive but biomechanically important: it allows the nervous system to modulate joint *impedance* independently of joint *torque*. This is a form of **variable impedance control** that has no direct analog in rigid robots with torque-controlled joints.

Position-dependent stiffness modulation is a nonlinear phenomenon central to the geometric analysis of affine control systems. Agrachev & Sachkov [1] develop the theory of feedback linearization and stabilization via nonlinear drift terms (Chapter 6), which encompasses exactly this scenario: using position-dependent actuation to achieve desired closed-loop dynamics.

5.2 The Mass Matrix of the Human Body

Computing the mass matrix $M(\mathbf{q})$ for a full musculoskeletal model follows the same recursive algorithms as Volume 0, Chapter 10 (the Composite Rigid Body Algorithm), using the segment inertial properties from Chapter 2. The key computational steps are:

1. Define the kinematic tree (segment connectivity and joint types)
2. Compute forward kinematics using the product of exponentials
3. Apply the CRBA to compute $M(\mathbf{q})$

For a 20-DOF model (pelvis + 2 legs + trunk + 2 arms), $M(\mathbf{q})$ is a 20×20 symmetric positive-definite matrix.

Exercises

1. **Bi-articular Energy Transfer.** For the gastrocnemius (crossing knee and ankle), show mathematically how knee extension can produce ankle torque through the bi-articular coupling.
2. **Co-Contraction Analysis.** Compute the joint stiffness when biceps and triceps co-contract at 50% activation with the elbow at 90° .
3. **Mass Matrix Properties.** For a 2-segment arm model, compute $M(\mathbf{q})$ at three configurations and verify positive-definiteness. Plot the condition number as a function of elbow angle.
4. **(Programming.)** Implement the CRBA for a 3-segment leg model using the de Leva segment properties. Compare with Pinocchio output.
5. **(Programming.)** Simulate a 2-DOF arm with 4 muscles (as in the code listing) performing a reaching movement. Use prescribed muscle activations and compute the resulting trajectory via forward dynamics.

Chapter 6

Inverse Problems in Biomechanics

In biomechanics, we almost never measure forces directly. We measure motion and infer the forces that caused it.

6.1 The Inverse Dynamics Pipeline

Inverse dynamics is the most widely used computational tool in biomechanics: given measured kinematics $\mathbf{q}(t)$, $\dot{\mathbf{q}}(t)$, $\ddot{\mathbf{q}}(t)$ and external forces $\mathbf{F}_{\text{ext}}(t)$, compute the net joint torques:

$$\boldsymbol{\tau}(t) = M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + g(\mathbf{q}) - J_c^T(\mathbf{q})\mathbf{F}_{\text{ext}}. \quad (6.1)$$

The standard pipeline comprises five stages:

1. **Data Collection:** Motion capture markers + force plates
2. **Marker Processing:** Filtering, gap-filling, labeling
3. **Inverse Kinematics:** Fit skeleton model to marker data
4. **Inverse Dynamics:** Compute joint torques from RNEA
5. **Muscle Force Estimation:** Distribute torques among muscles

6.1.1 Inverse Kinematics

Given marker positions $\mathbf{m}_i^{\text{meas}}(t) \in \mathbb{R}^3$ for $i = 1, \dots, N_m$ markers, inverse kinematics solves:

$$\mathbf{q}^*(t) = \arg \min_{\mathbf{q}} \sum_{i=1}^{N_m} w_i \left\| \mathbf{m}_i^{\text{model}}(\mathbf{q}) - \mathbf{m}_i^{\text{meas}}(t) \right\|^2, \quad (6.2)$$

where $\mathbf{m}_i^{\text{model}}(\mathbf{q})$ is the predicted marker position from the skeleton model and w_i are weighting factors.

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 def inverse_kinematics_frame(
5     marker_positions_measured: np.ndarray,
6     marker_positions_model_func,
```

```

7     q_initial: np.ndarray,
8     weights: np.ndarray | None = None
9 ) -> np.ndarray:
10     """Solve inverse kinematics for a single frame.
11
12     Parameters
13     -----
14     marker_positions_measured : np.ndarray, shape (n_markers, 3)
15         Measured marker positions.
16     marker_positions_model_func : callable
17         Function q -> (n_markers, 3) predicted marker positions.
18     q_initial : np.ndarray, shape (n_dof,)
19         Initial guess for joint angles.
20     weights : np.ndarray, shape (n_markers,), optional
21         Per-marker weights.
22
23     Returns
24     -----
25     np.ndarray, shape (n_dof,)
26         Optimal joint angles.
27     """
28     n_markers = marker_positions_measured.shape[0]
29     if weights is None:
30         weights = np.ones(n_markers)
31
32     def objective(q: np.ndarray) -> float:
33         m_model = marker_positions_model_func(q)
34         residuals = m_model - marker_positions_measured
35         return float(np.sum(weights[:, None] * residuals**2))
36
37     result = minimize(objective, q_initial, method='L-BFGS-B')
38     return result.x

```

Listing 6.1: Simplified inverse kinematics solver.

6.1.2 Bottom-Up vs. Top-Down Inverse Dynamics

Bottom-up: start from the ground contact (where forces are measured by force plates) and propagate upward through the kinematic chain. Most accurate when force plate data is available.

Top-down: start from the most distal segment and work inward. Useful when ground reaction forces are not available (e.g., swimming, throwing).

6.2 The Muscle Redundancy Problem

Inverse dynamics gives net joint torques $\boldsymbol{\tau} \in \mathbb{R}^n$, but we want individual muscle forces $\mathbf{F} \in \mathbb{R}^p$ with $p > n$ (typically $p = 3n$ to $5n$). This is an underdetermined system:

$$R(\mathbf{q})\mathbf{F} = \boldsymbol{\tau}, \quad \mathbf{F} \geq 0. \quad (6.3)$$

From a control-theoretic perspective, this is an *accessibility* problem. The system can reach a desired joint torque $\boldsymbol{\tau}$ if and only if $\boldsymbol{\tau}$ lies in the image of $R(\mathbf{q})$ —the reachable torque set.

When $p > n$, the system is over-actuated: multiple control inputs (muscle activation patterns) can produce the same output (joint torque). Agrachev & Sachkov [1], Chapter 3, develop this theory for general affine systems: the reachable set and controllability are characterized by Lie bracket conditions on the control vector fields, of which $R(\mathbf{q})$ is a concrete instance.

6.2.1 Static Optimization

The most common approach minimizes a cost function:

$$\min_{\mathbf{F}} \sum_{i=1}^p \left(\frac{F_i}{F_{0,i}} \right)^\alpha \quad \text{subject to} \quad R(\mathbf{q})\mathbf{F} = \boldsymbol{\tau}, \quad 0 \leq F_i \leq a_i \cdot f_L(\tilde{\ell}_i) \cdot F_{0,i}, \quad (6.4)$$

where $\alpha = 2$ (minimize sum of squared activations) or $\alpha = 3$ (minimize metabolic energy proxy). The constraint $F_i \leq a_i \cdot f_L \cdot F_{0,i}$ enforces the force-length and activation constraints from Chapter 3.

This cost-based resolution naturally emerges from geometric optimal control: Agrachev & Sachkov's framework [1], Chapters 5–6, shows that for over-actuated affine systems, optimal synthesis automatically selects among the infinitely many solutions to the constraint. The choice of cost function $\mathcal{L}(\mathbf{F})$ implicitly defines the reachable set and which solutions are ‘naturally’ preferred by the dynamics.

6.2.2 Computed Muscle Control (CMC)

CMC [?] combines forward dynamics with feedback control. It uses a PD controller to track the measured kinematics, with the controller output being the desired muscle excitations:

$$\ddot{\mathbf{q}}_{\text{des}} = \ddot{\mathbf{q}}_{\text{meas}} + K_v(\dot{\mathbf{q}}_{\text{meas}} - \dot{\mathbf{q}}) + K_p(\mathbf{q}_{\text{meas}} - \mathbf{q}), \quad (6.5)$$

where K_v and K_p are velocity and position feedback gains.

CMC is a tracking control law: it ensures that the system follows a reference trajectory despite the nonlinearity and redundancy of the muscle-driven system. Agrachev & Sachkov [1], Chapter 6, develop general feedback synthesis theory for nonlinear systems, showing that stability can be guaranteed via suitable Lyapunov functions and feedback design—exactly the principle underlying CMC's success.

6.3 Parameter Estimation

Many biomechanical parameters (segment masses, muscle strengths, joint axis locations) are not directly measurable and must be estimated from motion data.

6.3.1 Bayesian Framework

The parameter estimation problem is naturally Bayesian:

$$p(\boldsymbol{\theta}|\mathbf{y}) \propto p(\mathbf{y}|\boldsymbol{\theta}) \cdot p(\boldsymbol{\theta}), \quad (6.6)$$

where $\boldsymbol{\theta}$ are model parameters, $\mathbf{y}$ are observations, $p(\mathbf{y}|\boldsymbol{\theta})$ is the likelihood, and $p(\boldsymbol{\theta})$ is the prior (from literature values and physical constraints).

Exercises

1. **Inverse Dynamics.** For a 2-segment pendulum model of the arm, compute the shoulder and elbow torques required during a reaching movement given $\mathbf{q}(t) = (\sin(\pi t/T), \pi t/(2T))$.
2. **Muscle Redundancy.** For a single-DOF elbow with 3 muscles (biceps, brachialis, brachioradialis), solve the static optimization for a given elbow torque. Show that the solution depends on the cost function exponent α .
3. **(Programming.)** Implement the static optimization for a 2-DOF, 6-muscle system. Compare $\alpha = 2$ and $\alpha = 3$ solutions.
4. **(Programming.)** Implement an inverse kinematics solver for a 3-segment arm model with 7 markers. Test with synthetic marker data.

Chapter 7

Experimental Methods

The quality of a biomechanical analysis is limited by the quality of the measurements, not the sophistication of the model.

7.1 Motion Capture Systems

7.1.1 Marker-Based Systems

Optoelectronic motion capture systems (Vicon, Qualisys, OptiTrack) track retroreflective markers attached to the body surface. Standard configurations use 30–80 markers placed on bony landmarks following established protocols (Plug-in-Gait, Cleveland Clinic, ISB).

Key specifications for biomechanical applications:

- **Sampling rate:** 100–500 Hz for human motion (higher for impacts)
- **Spatial accuracy:** < 1 mm in calibrated volume
- **Marker size:** 9–14 mm diameter
- **Latency:** important for real-time applications (< 10 ms achievable)

7.1.2 Markerless Systems

Computer vision approaches (OpenPose, MediaPipe, DeepLabCut) estimate joint positions from video without physical markers. Accuracy is currently 10–30 mm, insufficient for clinical-grade inverse dynamics but useful for field measurements and large-scale studies.

```
1 import numpy as np
2 from scipy.signal import butter, filtfilt
3
4 def process_mocap_data(
5     marker_data: np.ndarray,
6     sample_rate: float,
7     cutoff_freq: float = 6.0,
8     filter_order: int = 4
9 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
10     """Process raw motion capture marker data.
11
12     Parameters
13     -----
```

```

14 marker_data : np.ndarray, shape (n_frames, n_markers, 3)
15     Raw marker positions in meters.
16 sample_rate : float
17     Sampling frequency in Hz.
18 cutoff_freq : float
19     Low-pass filter cutoff frequency in Hz.
20 filter_order : int
21     Butterworth filter order.
22
23 Returns
24 -----
25 tuple of np.ndarray
26     (positions, velocities, accelerations)
27     Each shape (n_frames, n_markers, 3).
28 """
29 # Design low-pass Butterworth filter
30 nyquist = sample_rate / 2.0
31 b, a = butter(filter_order, cutoff_freq / nyquist, btype='low')
32
33 n_frames, n_markers, _ = marker_data.shape
34 positions = np.zeros_like(marker_data)
35 velocities = np.zeros_like(marker_data)
36 accelerations = np.zeros_like(marker_data)
37
38 dt = 1.0 / sample_rate
39
40 for m in range(n_markers):
41     for axis in range(3):
42         # Filter position data
43         positions[:, m, axis] = filtfilt(
44             b, a, marker_data[:, m, axis]
45         )
46
47 # Central difference for velocities
48 velocities[1:-1] = (positions[2:] - positions[:-2]) / (2 * dt)
49 velocities[0] = velocities[1]
50 velocities[-1] = velocities[-2]
51
52 # Central difference for accelerations
53 accelerations[1:-1] = (
54     positions[2:] - 2 * positions[1:-1] + positions[:-2]
55 ) / dt**2
56 accelerations[0] = accelerations[1]
57 accelerations[-1] = accelerations[-2]
58
59 return positions, velocities, accelerations

```

Listing 7.1: Motion capture data processing pipeline.

7.2 Electromyography (EMG)

EMG measures the electrical activity of muscles, providing a window into neural commands. Surface EMG (sEMG) is non-invasive; intramuscular EMG (using fine-wire or needle electrodes) targets deep muscles.

7.2.1 EMG Processing

The raw EMG signal must be processed before use:

1. **Band-pass filter:** 20–450 Hz (remove movement artifact and noise)
2. **Full-wave rectification:** $|e(t)|$
3. **Envelope extraction:** low-pass filter at 3–6 Hz
4. **Normalization:** divide by maximum voluntary contraction (MVC) value

7.3 Force Plates

Force plates measure ground reaction forces (GRF) and moments. Standard specifications:

- 6-component measurement: $F_x, F_y, F_z, M_x, M_y, M_z$
- Sampling rate: 1000–2000 Hz
- Range: 0–5000 N vertical, 0–2500 N horizontal
- Center of pressure (CoP) accuracy: < 2 mm

The center of pressure is computed from force plate outputs:

$$x_{\text{CoP}} = -\frac{M_y + F_x \cdot d_z}{F_z}, \quad y_{\text{CoP}} = \frac{M_x - F_y \cdot d_z}{F_z}, \quad (7.1)$$

where d_z is the distance from the force plate surface to the transducer plane.

7.4 Inertial Measurement Units (IMUs)

IMUs combine accelerometers, gyroscopes, and magnetometers in a compact package. They provide:

- Angular velocity: $\boldsymbol{\omega}(t) \in \mathbb{R}^3$
- Linear acceleration: $\boldsymbol{a}(t) \in \mathbb{R}^3$
- Magnetic field: $\boldsymbol{B}(t) \in \mathbb{R}^3$ (for heading)

Orientation estimation from IMU data is a state estimation problem solvable using the extended Kalman filter (Volume I, Chapter 6) or complementary filters.

7.5 Data Synchronization

Multiple measurement systems must be temporally synchronized. Best practices:

- Hardware sync: trigger all systems from a common clock pulse
- Minimum: force plate and motion capture on the same clock
- EMG-to-motion delay: typically 10–80 ms (electromechanical delay)
- Cross-correlation for post-hoc alignment when hardware sync is unavailable

Exercises

1. **Filter Selection.** Using the provided code, filter a synthetic marker trajectory with cutoff frequencies of 4, 6, 8, and 12 Hz. Plot the effect on position, velocity, and acceleration. Discuss the trade-off between noise reduction and signal distortion.
2. **CoP Computation.** Given force plate data from a vertical jump (F_z peak = 3000 N), compute the center of pressure trajectory during takeoff.
3. **EMG Processing.** Process a synthetic EMG signal (sum of sinusoids + white noise) through the full processing pipeline. Plot each stage.
4. **(Programming.)** Implement an IMU orientation estimator using a complementary filter. Test with synthetic gyroscope and accelerometer data.

Chapter 8

Inference on Biological Systems

We cannot open the body and read the parameters. We must infer them from the traces left by motion.

8.1 Parameter Identification from Motion Data

Biological system parameters (muscle strengths, tendon slack lengths, segment inertias) cannot typically be measured directly in vivo. They must be inferred from observable motion data using optimization and estimation techniques.

8.1.1 Maximum Likelihood Estimation

Given a parameterized model $\dot{\mathbf{x}} = f(\mathbf{x}, \boldsymbol{\theta})$ with parameters $\boldsymbol{\theta}$ and noisy observations $\mathbf{y}_k = h(\mathbf{x}_k) + \mathbf{v}_k$ at times t_k , the maximum likelihood estimate is:

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \sum_{k=1}^N \|\mathbf{y}_k - h(\mathbf{x}_k(\boldsymbol{\theta}))\|_{R^{-1}}^2, \quad (8.1)$$

where R is the measurement noise covariance.

8.1.2 Bayesian Estimation with Physical Constraints

The Bayesian framework (introduced in Chapter 6) is particularly valuable for biomechanical parameter estimation because it naturally incorporates prior knowledge from the literature:

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 def log_posterior(
5     theta: np.ndarray,
6     observations: np.ndarray,
7     forward_model,
8     prior_mean: np.ndarray,
9     prior_cov: np.ndarray,
10    noise_cov: np.ndarray
11 ) -> float:
12     """Compute the log-posterior for parameter estimation.
13
14     Parameters
```

```

15 -----
16 theta : np.ndarray
17     Model parameters.
18 observations : np.ndarray, shape (n_obs, n_dims)
19     Observed data.
20 forward_model : callable
21     theta -> predicted_observations.
22 prior_mean : np.ndarray
23     Prior parameter mean.
24 prior_cov : np.ndarray
25     Prior parameter covariance.
26 noise_cov : np.ndarray
27     Observation noise covariance.
28
29 Returns
30 -----
31 float
32     Negative log-posterior (for minimization).
33 """
34 # Log-likelihood
35 predicted = forward_model(theta)
36 residuals = observations - predicted
37 R_inv = np.linalg.inv(noise_cov)
38 log_likelihood = -0.5 * np.sum(residuals @ R_inv * residuals)
39
40 # Log-prior
41 prior_residuals = theta - prior_mean
42 P_inv = np.linalg.inv(prior_cov)
43 log_prior = -0.5 * prior_residuals @ P_inv @ prior_residuals
44
45 return -(log_likelihood + log_prior)
46
47
48 def map_estimate(
49     observations: np.ndarray,
50     forward_model,
51     prior_mean: np.ndarray,
52     prior_cov: np.ndarray,
53     noise_cov: np.ndarray,
54     bounds: list[tuple[float, float]] | None = None
55 ) -> np.ndarray:
56     """Compute Maximum A Posteriori (MAP) parameter estimate.
57
58     Parameters
59     -----
60     observations : np.ndarray
61         Observed data.
62     forward_model : callable
63         Parameter-to-observation mapping.
64     prior_mean : np.ndarray
65         Prior mean from literature.
66     prior_cov : np.ndarray
67         Prior uncertainty.
68     noise_cov : np.ndarray
69         Measurement noise covariance.
70     bounds : list of tuple, optional

```

```

71     Physical bounds on parameters.
72
73     Returns
74     -----
75     np.ndarray
76         MAP parameter estimate.
77     """
78     result = minimize(
79         log_posterior,
80         prior_mean,
81         args=(observations, forward_model, prior_mean,
82             prior_cov, noise_cov),
83         method='L-BFGS-B',
84         bounds=bounds
85     )
86     return result.x

```

Listing 8.1: Bayesian parameter estimation for musculoskeletal model.

8.2 System Identification for Biological Systems

System identification techniques from control theory (Volume I) apply to biomechanical systems with modifications:

1. **Joint impedance identification:** Perturb the limb and measure the response. The stiffness, damping, and inertia of the joint can be estimated from the transfer function.
2. **Muscle parameter identification:** Optimize muscle parameters (F_0 , optimal length, tendon slack length) to match experimental force or torque data.
3. **Neural control identification:** Estimate the feedback gains used by the nervous system from perturbation experiments.

8.3 Machine Learning Approaches

Modern ML approaches to biomechanical inference include:

- Physics-informed neural networks (PINNs) that enforce equations of motion
- Gaussian process regression for joint angle estimation from sparse data
- Deep learning for markerless motion capture pose estimation

Exercises

1. **MAP Estimation.** Using the provided code, estimate the optimal fiber length of a biceps muscle from simulated isometric force data at 5 joint angles.
2. **Identifiability Analysis.** For a Hill-type muscle model, analyze which parameters can be uniquely estimated from isometric force-angle data alone. Which require dynamic data?

3. **(Programming.)** Implement a joint impedance identification experiment: simulate a perturbed pendulum and estimate stiffness and damping from the response.

Chapter 9

Deformable Bodies and Continuum Mechanics

When the rigid-body assumption fails, we do not abandon mechanics. We embrace the continuum.

9.1 When Rigid-Body Assumptions Fail

The rigid-body model fails when:

1. **Internal stresses are needed:** to predict injury, fracture, or tissue damage
2. **Deformation affects kinematics:** shaft flexibility in golf, spine compliance
3. **Wave propagation matters:** impact biomechanics, blast injury
4. **Fluid-structure interaction:** blood flow, synovial fluid, respiratory mechanics

9.2 Beam Models for Long Bones and Shafts

When deformation is small and the body is slender (length $\gg$ diameter), beam theory provides an efficient model. For a Euler-Bernoulli beam:

$$EI \frac{\partial^4 w}{\partial x^4} + \rho A \frac{\partial^2 w}{\partial t^2} = f(x, t), \quad (9.1)$$

where E is the elastic modulus, I is the second moment of area, ρ is density, A is cross-sectional area, $w(x, t)$ is transverse displacement, and f is the distributed load.

9.2.1 Modal Analysis

The solution is expanded in modes:

$$w(x, t) = \sum_{k=1}^N \phi_k(x) \eta_k(t), \quad (9.2)$$

where $\phi_k(x)$ are mode shapes and $\eta_k(t)$ are modal coordinates. This converts the PDE to a system of ODEs that can be appended to the rigid-body equations of motion.

9.2.2 Golf Club Shaft Flexibility

The golf club shaft is the most accessible example of a flexible link in sport biomechanics. A typical shaft has:

- Length: ~ 1.1 m
- First bending mode: 4–6 Hz (driver), 6–10 Hz (irons)
- Tip deflection at impact: 5–15 cm
- Lead/lag deflection: 2–5 cm
- Toe-down droop: 3–8 cm (gravity effect during downswing)

```

1 import numpy as np
2 from scipy.linalg import eigh
3
4 def shaft_modal_analysis(
5     n_elements: int = 20,
6     length: float = 1.1,
7     EI_root: float = 50.0,
8     EI_tip: float = 15.0,
9     rho_A: float = 0.15
10 ) -> tuple[np.ndarray, np.ndarray]:
11     """Compute natural frequencies and mode shapes of a tapered shaft.
12
13     Uses finite element beam model with linear EI taper.
14
15     Parameters
16     -----
17     n_elements : int
18         Number of finite elements.
19     length : float
20         Shaft length in meters.
21     EI_root : float
22         Flexural rigidity at root (grip end) in N*m2.
23     EI_tip : float
24         Flexural rigidity at tip (head end) in N*m2.
25     rho_A : float
26         Mass per unit length in kg/m.
27
28     Returns
29     -----
30     tuple of np.ndarray
31         (frequencies_hz, mode_shapes)
32     """
33     n_nodes = n_elements + 1
34     n_dof = 2 * n_nodes # displacement + rotation at each node
35     h = length / n_elements
36
37     K = np.zeros((n_dof, n_dof))
38     M = np.zeros((n_dof, n_dof))
39
40     for e in range(n_elements):
41         # Local EI for this element (linear taper)

```

```

42     xi = (e + 0.5) / n_elements
43     EI_local = EI_root + (EI_tip - EI_root) * xi
44
45     # Euler-Bernoulli beam element stiffness matrix
46     k_e = EI_local / h**3 * np.array([
47         [12, 6*h, -12, 6*h],
48         [6*h, 4*h**2, -6*h, 2*h**2],
49         [-12, -6*h, 12, -6*h],
50         [6*h, 2*h**2, -6*h, 4*h**2]
51     ])
52
53     # Consistent mass matrix
54     m_e = rho_A * h / 420 * np.array([
55         [156, 22*h, 54, -13*h],
56         [22*h, 4*h**2, 13*h, -3*h**2],
57         [54, 13*h, 156, -22*h],
58         [-13*h, -3*h**2, -22*h, 4*h**2]
59     ])
60
61     # Assembly
62     dofs = [2*e, 2*e+1, 2*e+2, 2*e+3]
63     for i_idx, i in enumerate(dofs):
64         for j_idx, j in enumerate(dofs):
65             K[i, j] += k_e[i_idx, j_idx]
66             M[i, j] += m_e[i_idx, j_idx]
67
68     # Boundary conditions: clamped at root (grip)
69     free_dofs = list(range(2, n_dof)) # Remove first node
70     K_free = K[np.ix_(free_dofs, free_dofs)]
71     M_free = M[np.ix_(free_dofs, free_dofs)]
72
73     # Solve eigenvalue problem
74     eigenvalues, eigenvectors = eigh(K_free, M_free)
75
76     # Convert to Hz
77     frequencies = np.sqrt(np.abs(eigenvalues)) / (2 * np.pi)
78
79     return frequencies[:6], eigenvectors[:, :6]
80
81
82 # Example
83 freqs, modes = shaft_modal_analysis()
84 print("Natural frequencies (Hz):")
85 for i, f in enumerate(freqs):
86     print(f" Mode {i+1}: {f:.1f} Hz")

```

Listing 9.1: Modal analysis of a golf club shaft.

9.3 Finite Element Methods (Overview)

For complex geometries (vertebral bodies, skull, articular surfaces), the finite element method (FEM) provides the most general approach. While a full treatment is beyond this text's scope, the key concept is:

The continuous domain Ω is divided into elements, and the displacement field is approximated using shape functions:

$$\mathbf{u}(\mathbf{x}, t) \approx \sum_i N_i(\mathbf{x}) \mathbf{u}_i(t), \quad (9.3)$$

leading to the assembled system:

$$M_{\text{FEM}} \ddot{\mathbf{u}} + C_{\text{FEM}} \dot{\mathbf{u}} + K_{\text{FEM}} \mathbf{u} = \mathbf{f}_{\text{ext}}. \quad (9.4)$$

Exercises

1. **Shaft Frequency.** Using the provided code, compute how the first natural frequency changes as the tip-to-root EI ratio varies from 0.1 to 0.9. Plot the result.
2. **Mode Shapes.** Plot the first three mode shapes of the shaft. Identify nodes (zero-crossings) and antinodes (maximum displacement).
3. **Timescale Separation.** If the golf downswing takes 0.25 s, estimate the number of oscillation cycles for each of the first 3 modes. Which modes are quasi-static?
4. **(Programming.)** Add a tip mass (clubhead, 0.2 kg) to the shaft FEM model and recompute the natural frequencies. How does the clubhead mass affect the frequencies?

Chapter 10

Applications of Control Theory to Biological Systems

The body is not a robot that happens to be made of meat. It is a system that has evolved to exploit its own physics.

10.1 Forward Dynamics Simulation

Forward dynamics simulation integrates the equations of motion given muscle excitations $\mathbf{u}(t)$:

$$\ddot{\mathbf{q}} = M(\mathbf{q})^{-1} [R(\mathbf{q})\mathbf{F}_{\text{muscle}} + J_c^T \mathbf{F}_c - C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} - g(\mathbf{q})], \quad (10.1)$$

coupled with the muscle state equations (Chapter 3):

$$\dot{a}_i = (u_i - a_i)/\tau_i, \quad (10.2)$$

$$\dot{\ell}_i^M = v_i^M(a_i, \ell_i^M, \ell_i^{MT}(\mathbf{q})). \quad (10.3)$$

```
1 import numpy as np
2 from scipy.integrate import solve_ivp
3 from typing import Callable
4
5 def forward_dynamics_simulation(
6     mass_matrix_func: Callable,
7     coriolis_func: Callable,
8     gravity_func: Callable,
9     moment_arm_func: Callable,
10    muscle_force_func: Callable,
11    excitation_func: Callable,
12    q0: np.ndarray,
13    qdot0: np.ndarray,
14    a0: np.ndarray,
15    t_span: tuple[float, float],
16    n_eval: int = 500
17 ) -> dict:
18     """Simulate forward dynamics of a musculoskeletal system.
19
20     Parameters
21     -----
22     mass_matrix_func : callable
23         q -> M(q), shape (n_dof, n_dof)
```

```

24 coriolis_func : callable
25     (q, qdot) -> C(q, qdot) @ qdot, shape (n_dof,)
26 gravity_func : callable
27     q -> g(q), shape (n_dof,)
28 moment_arm_func : callable
29     q -> R(q), shape (n_dof, n_muscles)
30 muscle_force_func : callable
31     (a, l_M, v_M) -> F_muscle for each muscle
32 excitation_func : callable
33     t -> u(t), shape (n_muscles,)
34 q0 : np.ndarray
35     Initial joint angles.
36 qdot0 : np.ndarray
37     Initial joint velocities.
38 a0 : np.ndarray
39     Initial muscle activations.
40 t_span : tuple
41     (t_start, t_end)
42 n_eval : int
43     Number of output time points.
44
45 Returns
46 -----
47 dict
48     Keys: 't', 'q', 'qdot', 'a', 'tau'
49 """
50 n_dof = len(q0)
51 n_muscles = len(a0)
52 t_eval = np.linspace(t_span[0], t_span[1], n_eval)
53
54 def dynamics(t: float, state: np.ndarray) -> np.ndarray:
55     q = state[:n_dof]
56     qdot = state[n_dof:2*n_dof]
57     a = np.clip(state[2*n_dof:], 0.0, 1.0)
58
59     # Muscle forces (simplified: assume known fiber length)
60     F_muscle = np.array([
61         a[i] * 500.0 # Simplified: F = a * F0
62         for i in range(n_muscles)
63     ])
64
65     # Joint torques
66     R = moment_arm_func(q)
67     tau = R @ F_muscle
68
69     # Skeletal dynamics
70     M = mass_matrix_func(q)
71     C_qdot = coriolis_func(q, qdot)
72     g = gravity_func(q)
73     qddot = np.linalg.solve(M, tau - C_qdot - g)
74
75     # Activation dynamics
76     u = excitation_func(t)
77     tau_act = 0.015 # 15 ms activation
78     tau_deact = 0.050 # 50 ms deactivation
79     adot = np.where(

```

```

80         u > a,
81         (u - a) / tau_act,
82         (u - a) / tau_deact
83     )
84
85     return np.concatenate([qdot, qddot, adot])
86
87     state0 = np.concatenate([q0, qdot0, a0])
88     sol = solve_ivp(dynamics, t_span, state0, t_eval=t_eval,
89                     method='RK45', max_step=0.001)
90
91     return {
92         't': sol.t,
93         'q': sol.y[:n_dof].T,
94         'qdot': sol.y[n_dof:2*n_dof].T,
95         'a': sol.y[2*n_dof:].T,
96     }

```

Listing 10.1: Forward dynamics integration for a musculoskeletal model.

10.2 Predictive Simulation

Predictive simulation uses trajectory optimization (Volume I, Chapter 5) to find the muscle excitations that produce a desired motion while minimizing a cost:

$$\min_{u(\cdot)} \int_0^T \mathcal{L}(\mathbf{x}, \mathbf{u}) dt \quad \text{s.t.} \quad \dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}), \mathbf{x}(0) = \mathbf{x}_0, h(\mathbf{x}, \mathbf{u}) \leq 0, \quad (10.4)$$

where $\mathcal{L}$ is a physiologically motivated cost (e.g., metabolic energy, muscle fatigue, deviation from desired kinematics) and h captures physical constraints (joint limits, muscle force capacity, contact constraints).

Agrachev & Sachkov [1], Theorem 8.1 (Pontryagin’s Maximum Principle), establishes the fundamental existence and optimality conditions for such optimal control problems. Their treatment covers affine systems with constraints and provides both theoretical guarantees and computational methods (the method of characteristics) for solving OCPs of this form.

10.3 Prosthetics and Exoskeleton Control

The control theory of Volumes I–II applies directly to assistive devices:

- **Powered prostheses:** trajectory tracking with impedance modulation
- **Exoskeletons:** assist-as-needed control using contraction theory
- **FES (Functional Electrical Stimulation):** direct muscle activation control for paralyzed limbs

10.4 Sport Biomechanics Applications

10.4.1 The Golf Swing as a Biomechanical System

The golf swing demonstrates every concept in this volume:

1. Multi-segment kinematic chain (15+ DOF)
2. Muscle-driven actuation with co-contraction
3. Bi-articular coupling (forearm muscles, lats)
4. Passive dynamics exploitation (wrist release)
5. Shaft flexibility (deformable body)
6. Ground reaction force coupling (closed chain at the feet)

The complete biomechanical analysis integrates:

- Motion capture (kinematics)
- Force plates (ground reaction forces)
- EMG (muscle activation patterns)
- Inverse dynamics (joint torques)
- Static optimization (muscle forces)
- Forward dynamics validation (predictive simulation)

10.5 Synthesis: From Theory to Application

The progression through this volume mirrors the progression through the series:

Volume	Concept	Biomechanical Application
Agrachev & Sachkov		
Volume	Concept	Biomechanical Application
Vol 0	Screw axes, PoE	Joint kinematics, ISA analysis
Vol I	Contraction theory	Stability of muscle-driven systems
Vol I	DDP/iLQR	Optimal muscle excitation patterns
Vol II	Trajectory tubes	Motor variability envelopes
Vol II	Phase variables	Gait cycle parameterization
Vol III	Hill model	Force prediction
Vol III	Inverse dynamics	Joint torque estimation
Vol IV	Motor control	Neural command generation

Exercises

1. **Forward Simulation.** Using the provided code (adapted for a simple 2-DOF model), simulate a reaching movement with prescribed muscle excitation ramps.
2. **Predictive vs. Tracking.** Compare the muscle activations from static optimization (tracking measured kinematics) with those from predictive simulation (optimizing a metabolic cost).
3. **Prosthetic Control.** Design an impedance controller for a prosthetic knee that provides different stiffness during stance and swing phases.
4. **(Programming.)** Implement a complete inverse dynamics $\rightarrow$ static optimization pipeline for a 3-DOF planar arm model with 6 muscles.

Bibliography

- [1] Andrei A. Agrachev and Yuri L. Sachkov. *Control Theory from the Geometric Viewpoint*, volume 87 of *Encyclopaedia of Mathematical Sciences*. Springer-Verlag, Berlin, 2004. Comprehensive treatment of geometric methods in control theory including Lie brackets, controllability via the Orbit theorem and Rashevskii–Chow theorem, the Pontryagin Maximum Principle from the symplectic viewpoint, sub-Riemannian geometry, and second-order optimality conditions. A foundational reference for the geometric approach to nonlinear control adopted throughout this series.